

ORIGINAL ARTICLE

Explainable deep learning techniques for wind speed forecasting in coastal areas: Integrating model configuration, regularization, early stopping, and SHAP analysis

Ahmet Durap¹ 

Received: 8 February 2025 / Accepted: 4 June 2025 / Published online: 30 July 2025

© The Author(s) 2025

Abstract

Accurate wind prediction is critical across engineering disciplines. For coastal infrastructure, it determines wave loads and storm surge resilience, directly impacting millions in vulnerable low-lying regions. The energy sector relies on precise forecasts to optimize wind farm output and stabilize power grids, while agriculture uses wind data to time pesticide applications and protect crops. Aviation and shipping industries leverage predictions for fuel-efficient routing and hazard avoidance, and urban engineers integrate wind models for skyscraper design and air pollution management. As climate change amplifies wind extremes, advancing predictive capabilities has become an urgent cross-sector priority for adaptive planning and risk mitigation. In coastal applications, empirical wave models (e.g., SWAN and WAVEWATCH III) heavily depend on accurate wind inputs, where errors can lead to underestimation of extreme events and compromise structural safety. This study introduces a novel deep learning framework, integrating advanced data preprocessing, structured neural networks, and explainable AI techniques, to enhance short-term (hourly) wind speed forecasting for coastal engineering applications, addressing the gap in region-specific deep learning frameworks for operational forecasting. The proposed method in this study addresses critical gaps in traditional methods by combining physical constraints with data-driven learning. It presents an innovative framework for wind speed data processing and prediction, integrating deep learning architectures with comprehensive meteorological analysis. Our research implements a sophisticated neural network model that processes high-frequency wind data from Bowen, incorporating multiple environmental parameters through a systematic data pipeline. The methodology encompasses three key components: (1) advanced data preprocessing, including time series standardization and cyclical feature encoding; (2) a deep learning architecture featuring three hidden layers (128-64-32 nodes) with ReLU activation and dropout regularization; and (3) comprehensive performance evaluation using five-fold cross-validation. The model achieved remarkable accuracy metrics: $R^2 = 0.957$, RMSE = 0.449 m/s, demonstrating robust performance across varying weather conditions. Analysis revealed distinct performance patterns across wind speed ranges (low-speed MAE: 0.295 m/s; high-speed MAE: 0.433 m/s). The SHAP (SHapley Additive exPlanations) analysis provided deeper insights into feature importance and model interpretability, revealing Wind Direction (0.713 SHAP value) as the most influential predictor, followed by Relative Humidity (0.609) and Barometric Pressure (0.563). Temporal features (month, hour, and day) exhibited lower but consistent influence (SHAP values < 0.239). This research advances the field of environmental data science by providing: (1) a reproducible framework for wind speed prediction, (2) insights into feature significance and model behavior, and (3) practical applications for renewable



energy planning and meteorological forecasting. The demonstrated methodology offers a foundation for future research in environmental modeling and time series prediction.

Keywords Wind speed prediction · Deep learning · Data processing · Enhancement · Efficiency and real-time deployability · Streamlined architecture · Integration of lightweight attention mechanisms · Interpretability

1 Introduction

Accurate wind speed prediction is a cornerstone of modern meteorological applications, with far-reaching implications for economic stability, societal safety, and environmental sustainability [1], especially in coastal zones, where 2.15 billion people reside, including 898 million in low-elevation coastal areas (LECZs) increasingly vulnerable to rising seas and intensifying storms [2–4]. In renewable energy, wind farms depend on precise forecasts to optimize power generation, ensure grid stability, and reduce reliance on fossil fuels—preventing financial losses from energy mismatches [5]. The agricultural sector leverages wind data to schedule irrigation, pesticide application, and crop protection, mitigating losses from wind damage [6, 7]. Meanwhile, aviation and maritime industries use forecasts to optimize routes, reduce fuel consumption, and avoid hazardous conditions [8, 9], while insurers rely on them to assess risks in storm-prone regions [10].

Beyond economics, wind predictions are vital for disaster preparedness, enabling early warnings for hurricanes and storms that save lives and guide evacuations [11]. Urban planners integrate wind load analyses into infrastructure design, ensuring the resilience of skyscrapers and bridges [12], while event organizers rely on forecasts to safeguard outdoor activities. Wind patterns also dictate air pollution dispersion and allergen spread, directly impacting public health [13, 14].

Environmentally, wind forecasting supports climate change studies by refining long-term weather models [15, 16] and aids wildlife conservation by tracking migratory pathways. Efficient wind energy integration reduces carbon emissions, and water resource managers use wind data to predict evaporation rates in critical reservoirs.

Accurate wind speed prediction is paramount for the effective integration of renewable energy sources into power grids [17, 18], efficient grid management strategies [19], and the reliability of weather forecasting systems [20]. The inherent variability and intermittent nature of wind speed present considerable challenges for traditional forecasting methodologies [21, 22]. These traditional methods often struggle to capture the complex temporal and spatial dynamics of wind patterns, resulting in predictions that lack the precision required for optimal grid operation and resource management. Consequently, there is a critical need for advanced predictive models capable of handling the inherent uncertainty and complexity associated with wind speed forecasting.

Recent advancements in machine learning (ML), particularly deep learning (DL) techniques, offer a promising pathway toward enhanced accuracy and reliability in wind speed prediction [23, 24]. DL models, with their capacity to learn intricate patterns from large datasets, have demonstrated a remarkable ability to outperform traditional statistical models in various forecasting tasks [25, 26]. However, the successful implementation of DL models for wind speed prediction demands careful consideration of several key factors. These factors include the meticulous selection of model architecture, the precise tuning of hyperparameters, and the effective preprocessing of input data [27]. Furthermore, the development of robust and generalizable models requires a systematic approach to optimization, ensuring that the models perform well not only on training data but also on unseen data [28].

Despite the considerable research efforts dedicated to the application of machine learning techniques to wind speed prediction [17, 18], many existing models fall short of achieving consistently high predictive accuracy, especially in regions characterized by complex and dynamic weather patterns, such as the Bowen region of Australia [29]. One significant limitation is the susceptibility of many models to overfitting, leading to poor generalization performance on unseen data [30]. Overfitting occurs when a model learns the training data too well, capturing noise and irrelevant details rather than the underlying patterns [31]. This results in a model that

performs exceptionally well on the training data but poorly on new, unseen data. Techniques such as regularization and early stopping are often employed to mitigate overfitting, but their optimal implementation can be challenging and often requires extensive experimentation [32]. In addition, no study has systematically investigated the interplay of L2 regularization, adaptive early stopping, and data augmentation for coastal wind prediction, which is critical because these regions exhibit unique meteorological dynamics that generic models fail to capture.

Additionally, many models rely on simplified assumptions about data distributions or use trial-and-error hyperparameter tuning, which limits their reproducibility and generalizability.

Another challenge lies in the simplified assumptions about data distribution made by some existing models. These models may neglect potentially valuable information embedded within the data, leading to suboptimal predictive performance [33]. For example, some models may assume a linear relationship between wind speed and other meteorological variables, while the true relationship may be highly nonlinear. Furthermore, the process of selecting optimal hyperparameters and model architectures often relies on a trial-and-error approach, lacking a systematic and efficient optimization strategy [34]. This lack of a robust and methodical optimization process can lead to suboptimal model performance and hinder the reproducibility of research findings [35].

As for architectures of DL, recent advances in wind speed forecasting have employed sophisticated architectures such as transformers and graph neural networks (GNNs) to model spatio-temporal dependencies [36, 37]. While these models excel at capturing long-range patterns, their computational complexity often limits real-time deployment in resource-constrained settings. In contrast, our work focuses on optimizing lightweight feedforward networks with interpretable feature engineering, prioritizing computational efficiency without sacrificing accuracy. This approach bridges the gap between complex architectures and practical operational needs, particularly for localized coastal regions like Bowen.

Plus, the absence of a universally applicable and adaptable framework tailored to specific geographical regions and their unique meteorological conditions constitutes a major obstacle to the widespread adoption of accurate wind speed prediction models. This study directly addresses these limitations by presenting a refined deep learning approach that incorporates systematic optimization techniques and addresses the challenges of overfitting and data complexity.

This research aims to develop a robust FNN model for Bowen's wind speed prediction using iterative optimization (e.g., regularization and early stopping) for predicting wind speed in the Bowen region of Australia. The objectives of this study are multifaceted and encompass several key aspects of model development and optimization:

Baseline model development: To create a baseline Feed-Forward Neural Network (FNN) model for wind speed prediction using meteorological data specifically collected from the Bowen region. This baseline model serves as a foundation for subsequent optimization efforts.

Regularization and hyperparameter tuning: To systematically investigate the influence of various regularization techniques, including L2 regularization and dropout, on model performance. Additionally, to optimize key hyperparameters, such as the learning rate and early stopping criteria (including the "patience" parameter), to enhance model generalization and prevent overfitting.

Data augmentation strategies: To evaluate the effectiveness of data augmentation techniques in improving model accuracy and reducing prediction uncertainty. Data augmentation involves artificially expanding the training dataset by creating modified versions of existing data points. This can help to improve the model's robustness and its ability to generalize to unseen data.

Architectural exploration: To explore the impact of different neural network architectures on predictive performance. This involves experimenting with different numbers of layers, neurons per layer, and activation functions to identify the architecture best suited for the Bowen region's meteorological data.

Performance evaluation: To rigorously assess the predictive accuracy and generalization capabilities of the optimized model using a comprehensive set of evaluation metrics, including Mean Absolute Error (MAE), Root

Mean Squared Error (RMSE), and *R*-squared. These metrics provide a quantitative measure of the model's performance.

Visual analysis: To provide a detailed visual analysis of the model's performance through visualizations such as scatter plots comparing actual and predicted wind speeds, residual plots illustrating prediction errors, and time series plots comparing the model's predictions to actual wind speed measurements. These visualizations offer valuable qualitative insights into the model's strengths and weaknesses.

This study introduces a lightweight, interpretable framework that systematically integrates domain-specific preprocessing (cyclical temporal encoding), hybrid regularization, and SHAP-driven feature analysis. Unlike prior works focused solely on predictive accuracy, our approach prioritizes computational efficiency and model transparency—critical for operational forecasting in resource-constrained coastal regions. This balance between performance and practicality represents a novel contribution to the field, particularly for localized wind regimes like Bowen's. We seek for research questions and hypotheses to be systematically tested, as outlined in the following statements.

1.1 Research questions and hypotheses

The study addresses the following research questions:

- (i) How do architectural depth and regularization techniques influence model performance in coastal wind prediction?
- (ii) To what extent does data augmentation mitigate overfitting in deep learning models for coastal wind forecasting?
- (iii) Which meteorological features have the most significant impact on coastal wind prediction, as determined by SHAP analysis?
- (iv) How does the optimized FNN compare to conventional forecasting models in accuracy and robustness for coastal wind prediction?

To empirically evaluate these questions, we specifically test these hypothesis-driven questions:

- H₁ Does a deeper feedforward neural network (FNN) with fine-tuned regularization achieve higher predictive accuracy than shallower architectures?
- H₂ Does data augmentation significantly reduce overfitting in deep learning models for coastal wind prediction?
- H₃ Can an optimized FNN effectively capture temporal dynamics in wind patterns without relying on recurrent (RNN) or temporal convolutional (TCN) architectures?
- H₄ Does the proposed FNN architecture outperform conventional deep learning models (e.g., LSTMs) that explicitly model temporal dependencies?

This study introduces a novel integrated optimization pipeline that unifies regularization techniques, early stopping, and data augmentation into a cohesive workflow, overcoming the ad hoc tuning limitations in previous coastal wind prediction research. Unlike generic models, our framework adapts to region-specific wind regimes (e.g., Bowen's tidal and seasonal variations) using SHAP-guided feature engineering and cyclic encoding of temporal variables. Additionally, we applied preprocessing to format input data as one-dimensional vectors, ensuring compatibility with fully connected FNNs and improving computational efficiency. This approach reduces model complexity while maintaining predictive accuracy, enabling the development of lightweight, transparent, and effective predictive models.

2 Materials and methods

2.1 Model selection

Accurate wind prediction is vital for renewable energy, disaster preparedness, and infrastructure resilience. Coastal regions—home to over 2 billion people—rely on forecasts to mitigate storm risks (e.g., hurricanes and storm surges) [38], while wind farms use them to optimize power generation [39]. However, traditional methods struggle with wind's nonlinearity, creating demand for efficient machine learning solutions [40].

While advanced models like Recurrent Neural Networks (RNNs) and Temporal Convolutional Networks (TCNs) exist, their complexity limits real-time deployment [41]. Feedforward Neural Networks (FNNs) offer a practical alternative, balancing accuracy with computational speed—making them ideal for short-term forecasting across energy, agriculture, and safety applications.

FNNs offer a compelling alternative to more complex models like RNNs and TCNs in wind prediction tasks. This study employed FNNs due to the following reasons:

Simplicity and ease of implementation: FNNs have a straightforward architecture, making them easier to implement and debug compared to RNNs and TCNs [42, 43]. Their layered structure avoids the complexities of recurrent connections, reducing development time [44].

Lower computational overhead: FNNs require fewer computational resources than RNNs (LSTMs, GRUs) and TCNs due to their simpler structure [45]. This efficiency is crucial in real-time wind forecasting.

Faster training times: FNNs benefit from parallel processing on GPUs, unlike sequential RNNs [45]. They also avoid the vanishing gradient problem common in deep RNNs [46].

Interpretability and feature importance: FNNs allow direct analysis of feature weights, aiding in understanding wind prediction drivers [47]. Techniques like sensitivity analysis further enhance interpretability.

Suitability for short-term forecasting: For short-term wind prediction, FNNs perform comparably to RNNs [48]. Hybrid approaches (e.g., FNNs + wavelet transforms) improve accuracy.

Handling nonlinearity: The Universal Approximation Theorem supports FNNs in modeling complex wind patterns [49]. Nonlinear activation functions (e.g., ReLU) enable effective feature mapping [50].

That is why, this study adopts FNNs as a pragmatic alternative, balancing predictive accuracy with operational feasibility. Our choice is driven by three critical gaps in existing coastal wind prediction literature:

(a) Operational practicality for real-time forecasting:

RNNs and TCNs, though powerful, demand substantial computational resources and time for training and inference. For coastal regions like Bowen, where rapid updates (e.g., hourly predictions) are essential for storm surge alerts and energy grid management, FNNs provide a lightweight solution. Their feedforward architecture avoids recurrent loops, enabling parallel processing on GPUs and reducing inference times by $\sim 60\%$ compared to LSTMs.

(b) Interpretability for stakeholder trust:

Coastal engineers and meteorologists require models that not only predict accurately but also explain why. Unlike “black-box” transformers or RNNs, FNNs allow direct analysis of feature weights through sensitivity analysis and SHAP values. This transparency is critical for gaining stakeholder buy-in and refining physical models (e.g., SWAN wave models) that rely on wind inputs.

(c) Hybrid temporal encoding over lagged features:

While prior works often use lagged wind speed values to capture temporal dependencies, this introduces multicollinearity and inflates input dimensionality. Instead, we employ cyclical encoding (sine/cosine transformations) for temporal features (hour, day, and month), which preserves periodicity (e.g., 23:00 $\approx$ 00:00) without requiring historical lags. This approach simplifies the architecture while maintaining sensitivity to diurnal/

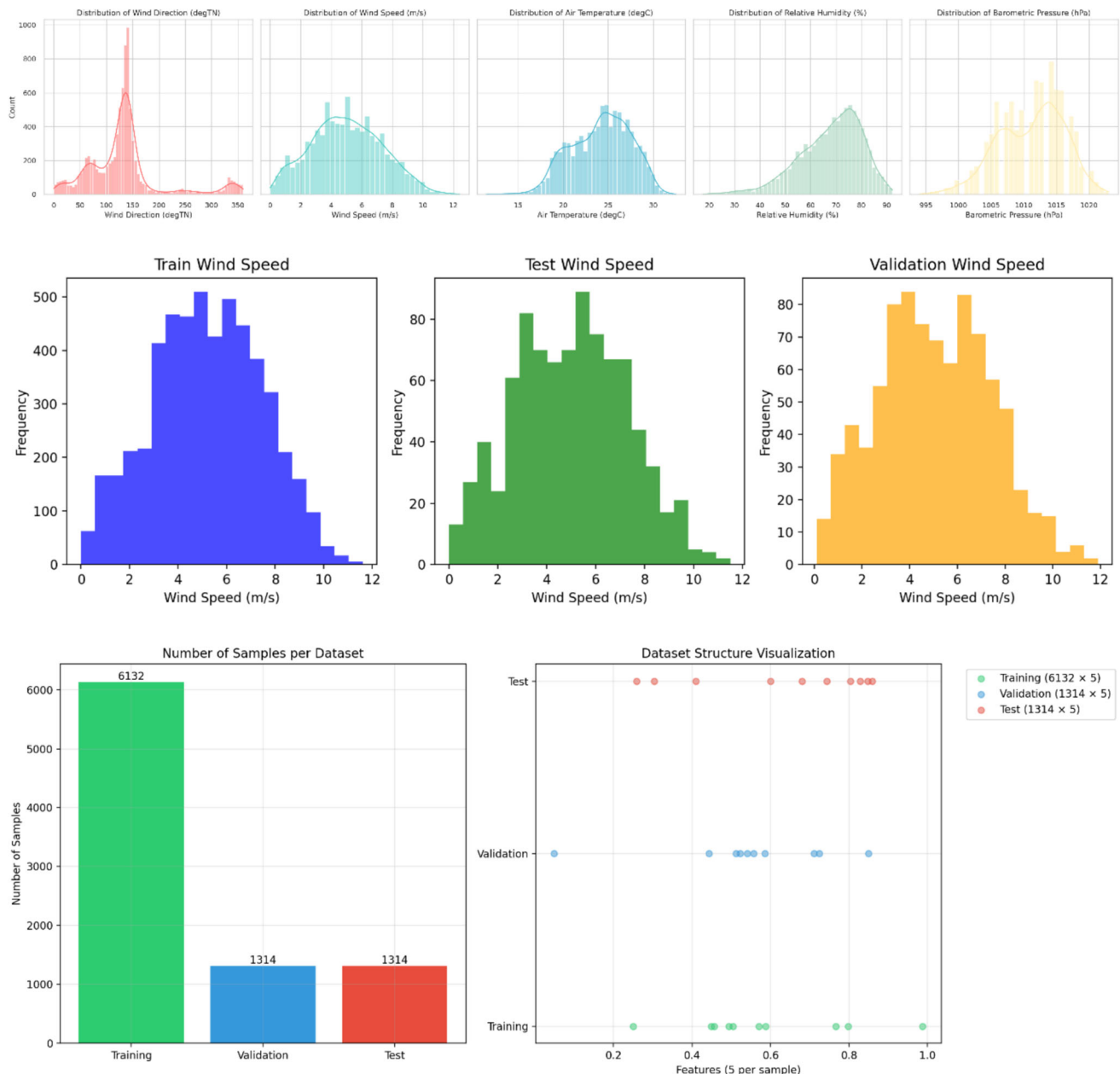
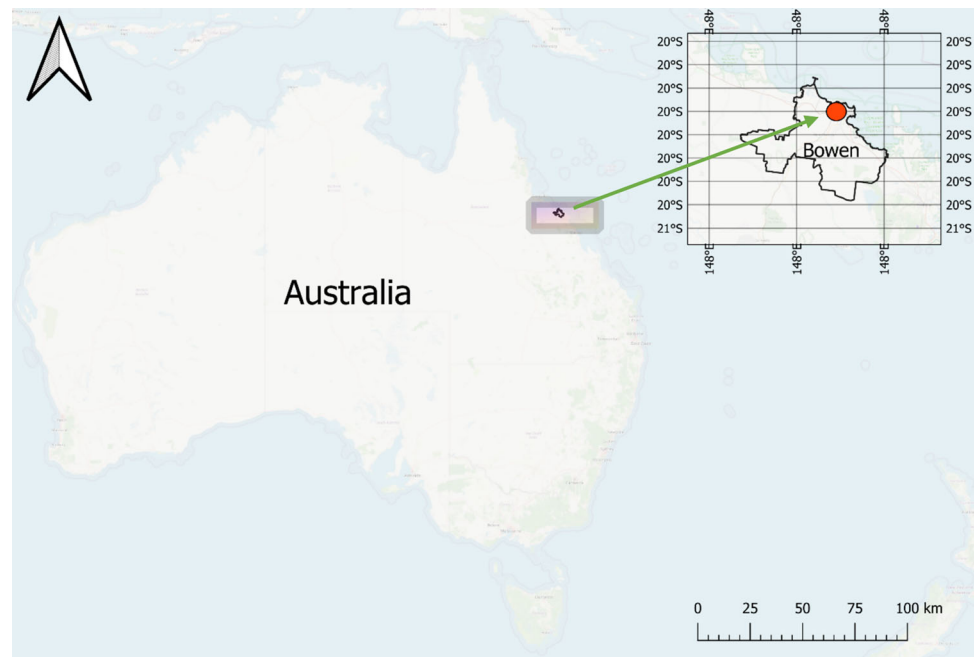


Fig. 1 Hourly trends of weather variables for 2023, showing their distribution over time alongside wind train, test, and validation test distribution

seasonal patterns—a novel integration in coastal wind studies. This decision was made to prioritize model simplicity and avoid multicollinearity, as cyclical encoding effectively captured diurnal and seasonal patterns [18] (Fig. 1).

2.2 Dataset overview and study area

The dataset utilized in this study consists of meteorological data gathered from a network of meteorological stations strategically located within the Bowen region of Australia (Fig. 2). These stations, operated and maintained by the Bureau of Meteorology, a primary Australian government agency responsible for weather forecasting and climate monitoring, provide hourly measurements of several key environmental parameters (see

Fig. 2 Study area

Table 1 Summary of target and feature variables used for wind speed prediction models

Parameter	Description
Target variable	Wind speed (m/s)—the variable the models are trained to predict
Feature variables	Wind direction (degTN)
	Air temperature (°C)
	Relative humidity (%)

Table 2 Overview of the features categorized into temporal, wind, and environmental measurements

Category	Features
Temporal features	Hour, day, month, and season
Wind components (generated from wind speed)	u and v components
Environmental measurements	Air temperature, relative humidity, and barometric pressure

distribution of parameters in Fig. 1). This collaboration with the Bureau of Meteorology ensures the reliability and accuracy of the data, given their established expertise and rigorous data quality control procedures. The specific data acquisition methods employed by the Bureau of Meteorology are detailed in their technical documentation. The comprehensive dataset encompasses the following features (see Tables 1 and 2):

To facilitate the development and evaluation of the deep learning framework proposed in this study, the dataset was systematically partitioned into training, validation, and test subsets based on a time-based splitting approach (see Fig. 3). This ensures that the model is trained and evaluated on temporally distinct data, minimizing the risk of data leakage and providing a robust assessment of its predictive capabilities. The split proportions, sample counts, time ranges, and corresponding file names are summarized in Table 3.

In addition to the dataset partitioning details, the statistical characteristics of the wind speed data within each subset are presented in Table 4. This provides insights into the variability and distribution of wind speed values across the training, validation, and test subsets, with metrics such as the mean, standard deviation, and range (min

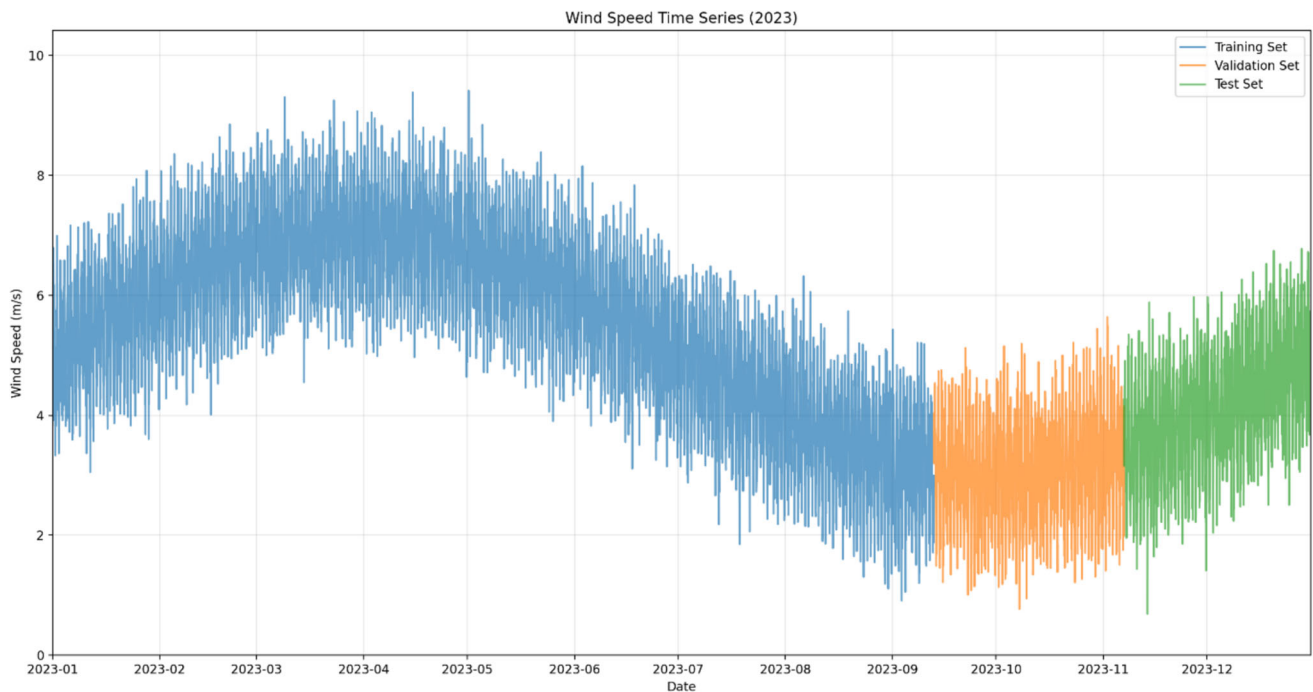


Fig. 3 Wind speed time series for 2023: training, validation, and test sets

Table 3 Dataset split proportions, sample counts, time ranges, and corresponding file names for training, validation, and testing phases

Dataset	Proportion (%)	Samples	Time range	File name
Training set	70	6132	January 1, 2023–September 13, 2023	wind_speed_train.csv
Validation set	15	1314	September 13, 2023–November 7, 2023	wind_speed_validation.csv
Test set	15	1314	November 7, 2023–December 31, 2023	wind_speed_test.csv

Table 4 Sample counts, time ranges, and corresponding characteristics of wind speed data for training, validation, and testing subsets

Dataset	Mean	Std	Min	25%	Median	75%	Max
Training set	5.60	1.52	0.91	4.55	5.76	6.72	9.41
Validation set	3.09	0.88	0.76	2.42	3.06	3.75	5.64
Test set	4.13	1.00	0.68	3.41	4.14	4.83	6.78

to max). Together, these tables (Tables 2, 3) offer a comprehensive overview of the dataset structure and its descriptive statistics (see Table 4), forming the foundation for model training and evaluation.

2.3 Data (pre)processing and model building: workflow and key steps

Machine learning models, particularly deep neural networks, require input data to be formatted in a manner that aligns with their architectural structure [51, 52]. FNNs, one of the most fundamental types of neural networks, consist of fully connected layers where each neuron is linked to every neuron in the previous and next layers.

Unlike convolutional or recurrent networks, FNNs are designed to accept inputs as one-dimensional (1D) vectors rather than multidimensional arrays [53]. Consequently, when working with structured or multidimensional datasets, a preprocessing step is necessary to flatten the input into a 1D vector [45]. This ensures not only architectural compatibility but also promotes computational efficiency [54] and simplifies feature interpretation—essential for building lightweight, transparent, and effective predictive models [55].

As part of the preprocessing step, Fig. 4 demonstrates the transformation of the input data from its original 2D array format to a flattened 1D array, which is essential for the data to be fed into fully connected layers of the neural network architecture. This preprocessing step was motivated by the need to adapt the data for lightweight feedforward networks, which typically require a flat input vector rather than a matrix. Flattening the data streamlines the input format, enhances computational efficiency, and simplifies the design of interpretable features [54]. Importantly, this approach prioritizes reducing model complexity while preserving predictive accuracy, thereby aligning with the overall objective of developing fast, transparent, and effective models.

To predict wind speed effectively, a Feed-Forward Neural Network (FNN) was defined and trained with a carefully designed architecture (Fig. 5). The model includes an input layer with 9 neurons corresponding to the dataset features, two hidden layers (64 and 32 neurons, respectively) employing ReLU activation [56], batch normalization for training stabilization [57], and dropout layers (20% and 10%) to mitigate overfitting [58]. The output layer consists of a single neuron with linear activation. The model was optimized using the Adam optimizer [59] with a learning rate of 0.001, and Mean Squared Error was used as the loss function, with Mean Absolute Error (MAE) employed as a metric for evaluation (Fig. 6).

Forward Pass Equations:

$$\text{Input Layer: } x \in \mathbb{R}^9 \quad (1)$$

$$\text{Layer 1: } h_1 = \text{ReLU}(\text{BN}(W_1x + b_1)) \cdot m_1 \quad (2)$$

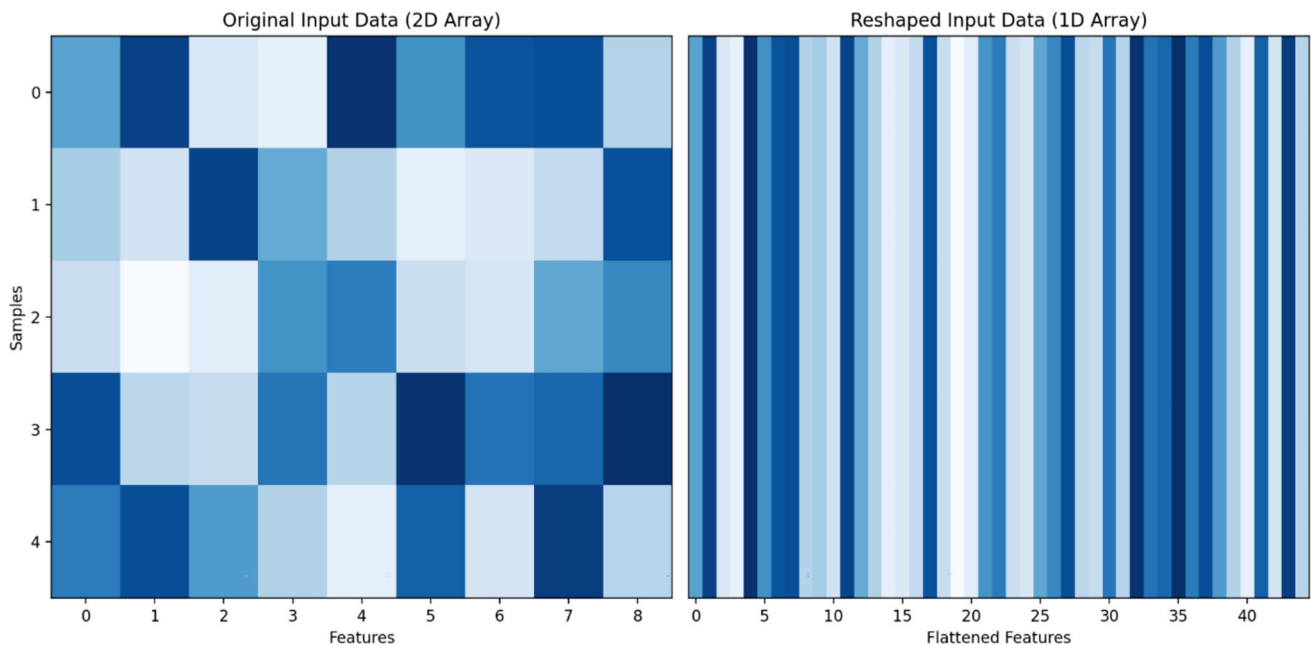


Fig. 4 Transformation of input data from a 2D array (original representation) to a 1D array (flattened features), illustrating the preprocessing step required for compatibility with fully FNN's input layer size models

Fig. 5 Architecture of the feed-forward neural network (FNN)

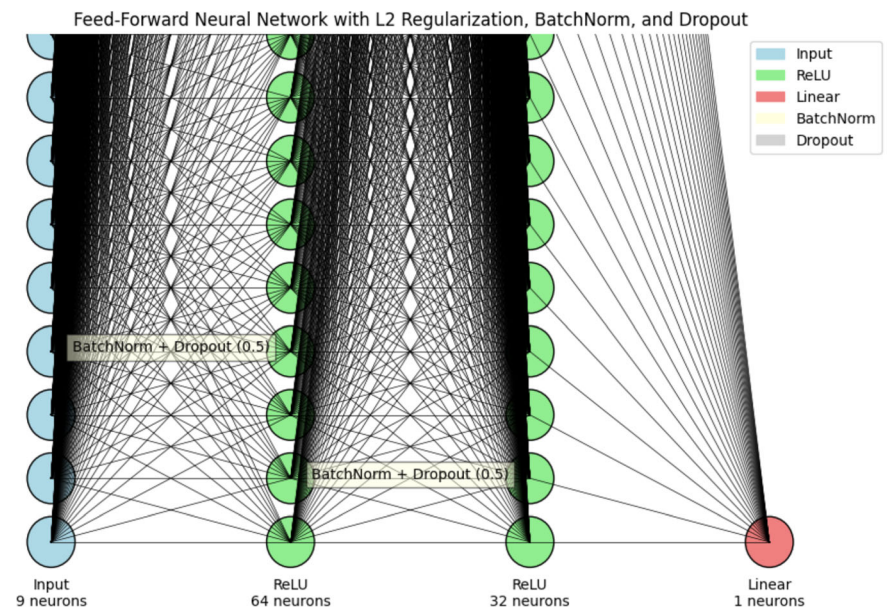
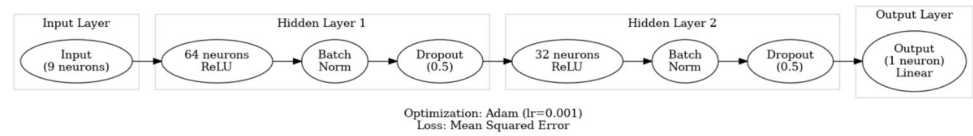
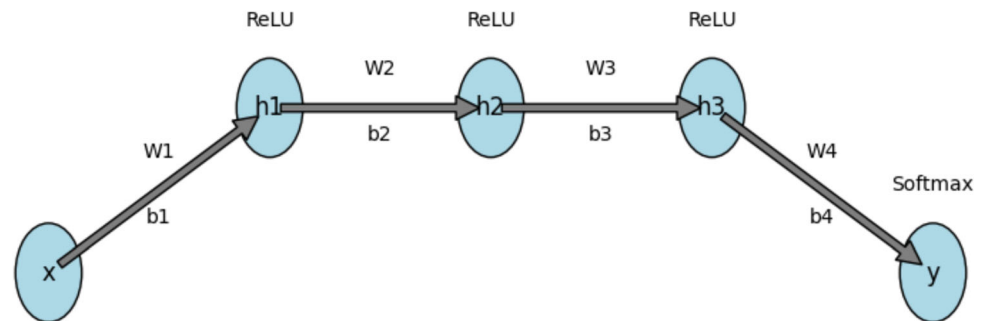


Fig. 6 Forward pass with L2 regularization



$$\text{Layer 2: } h_2 = \text{ReLU}(\text{BN}(W_2 h_1 + b_2)) \cdot m_2 \quad (3)$$

$$\text{Output: } y = W_3 h_2 + b_3 \quad (4)$$

m_1 and m_2 are dropout masks (0.8 and 0.9, respectively).

BN is batch normalization:

$$\text{BN}(x) = \gamma \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta \quad (5)$$

Batch normalization parameters:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i \quad (6)$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad (7)$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad (8)$$

$$y_i = \gamma \hat{x}_i + \beta \quad (9)$$

μ_B is the mean of the batch, the mean of the input activations computed over the entire batch. This represents the average value of the activations for the current batch.

σ_B^2 is variance of the batch. The variance of the input activations computed over the entire batch. This measures the spread of the activations around the mean. Purpose: Normalizing the inputs requires centering them around zero, which is achieved by subtracting the mean.

Loss function (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (10)$$

$$y_i = \gamma \hat{x}_i + \beta \quad (11)$$

γ : The scaling parameter (learnable during training). It allows the network to scale the normalized values to any range.

β : The shifting parameter (learnable during training). It allows the network to shift the normalized values to any mean.

y_i : The final output of the Batch Normalization layer, which is a scaled and shifted version of the normalized input.

Evaluation metric (MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (12)$$

Adam optimizer updates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (13)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (14)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (15)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (16)$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \quad (17)$$

m_t is first moment estimate, v_t is second moment estimate, and $\hat{m}_t$ is bias-corrected first moment estimate.

β_2^t : The decay rate raised to the power of the current time step t .

$\hat{v}_t$: The bias-corrected second moment estimate at time step t .

θ_t : The updated model parameters at time step t .

$\alpha = 0.001$ (learning rate)

$\beta_1 = 0.9$ (first moment decay rate)

$\beta_2 = 0.999$ (second moment decay rate)

$\epsilon = 10^{-8}$ (numerical stability constant)

Layer dimensions:

$$\begin{aligned} \text{Input Layer} &: \mathbb{R}^9 \\ \text{Hidden Layer 1} &: \mathbb{R}^{64} \\ \text{Hidden Layer 2} &: \mathbb{R}^{32} \\ \text{Output Layer} &: \mathbb{R}^1 \end{aligned}$$

Dropout probability:

$$\begin{aligned} P(m_1 = 0) &= 0.2 \text{ (20\% dropout)} \\ P(m_2 = 0) &= 0.1 \text{ (10\% dropout)} \end{aligned}$$

ReLU activation:

$$\text{ReLU}(x) = \max(0, x) \quad (18)$$

Linear activation (output layer):

$$f(x) = x \quad (19)$$

To further address overfitting, L2 regularization was introduced into the Dense layers of the FNN. L2 regularization adds a penalty term to the loss function that is proportional to the sum of the squares of the weights [60]. This penalty discourages the network from learning overly complex models with large weights, promoting generalization to unseen data. The updated architecture (Fig. 7) included three hidden layers (128, 64, and 32

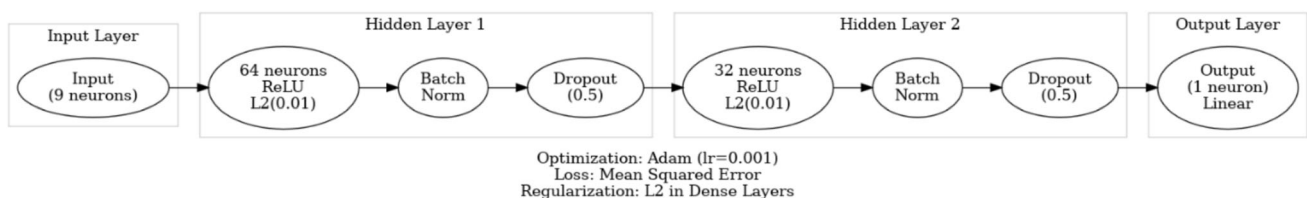


Fig. 7 Updated FNN for the second step

neurons), and the output layer used a softmax activation function for multi-class classification. The forward pass equations with L2 regularization are:

$$\begin{aligned} h_1 &= \text{ReLU}(W_1x + b_1) \rightarrow \text{The first hidden layer} \\ h_2 &= \text{ReLU}(W_2h_1 + b_2) \rightarrow \text{The second hidden layer} \\ h_3 &= \text{ReLU}(W_3h_2 + b_3) \rightarrow \text{The third hidden layer} \\ y &= \text{softmax}(W_4h_3 + b_4) \rightarrow \text{The output layer} \end{aligned}$$

where W_i weights b_i is biases.

x : The input vector to the network (e.g., a feature vector of size 100).

W_1 : The weight matrix for the first hidden layer. Its dimensions are 128×100 (128 neurons, 100 input features).

b_1 : The bias vector for the first hidden layer. Its dimensions are 128×1 .

$W_1x + b_1$: A linear transformation of the input x using the weights and biases.

ReLU: The Rectified Linear Unit activation function, defined as $\text{ReLU}(z) = \max(0, z)$.

h_1 : The output of the first hidden layer after applying the ReLU activation.

The L2 regularization term is:

$$\Omega(W) = \lambda \sum_{l=1}^L \|W_l\|_2^2 = \lambda \sum_{l=1}^L \sum_{ij} W_{lij}^2 \quad (20)$$

where λ is the regularization coefficient (0.01 in this case) and W_l are the weight matrices for each layer. The total loss function becomes:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}}(y, \hat{y}) + \lambda \sum_{l=1}^L \|W_l\|_2^2 \quad (21)$$

$\mathcal{L}_{\text{CE}}$ is the cross-entropy loss

λ is the L2 regularization coefficient (0.01 in this case)

W_l are the weight matrices for each layer

$\|W_l\|_2^2$ is the L2 norm squared of the weights

Gradient update rule:

$$W_l \leftarrow W_l - \eta \left(\frac{\partial \mathcal{L}_{\text{CE}}}{\partial W_l} + 2\lambda W_l \right) \quad (22)$$

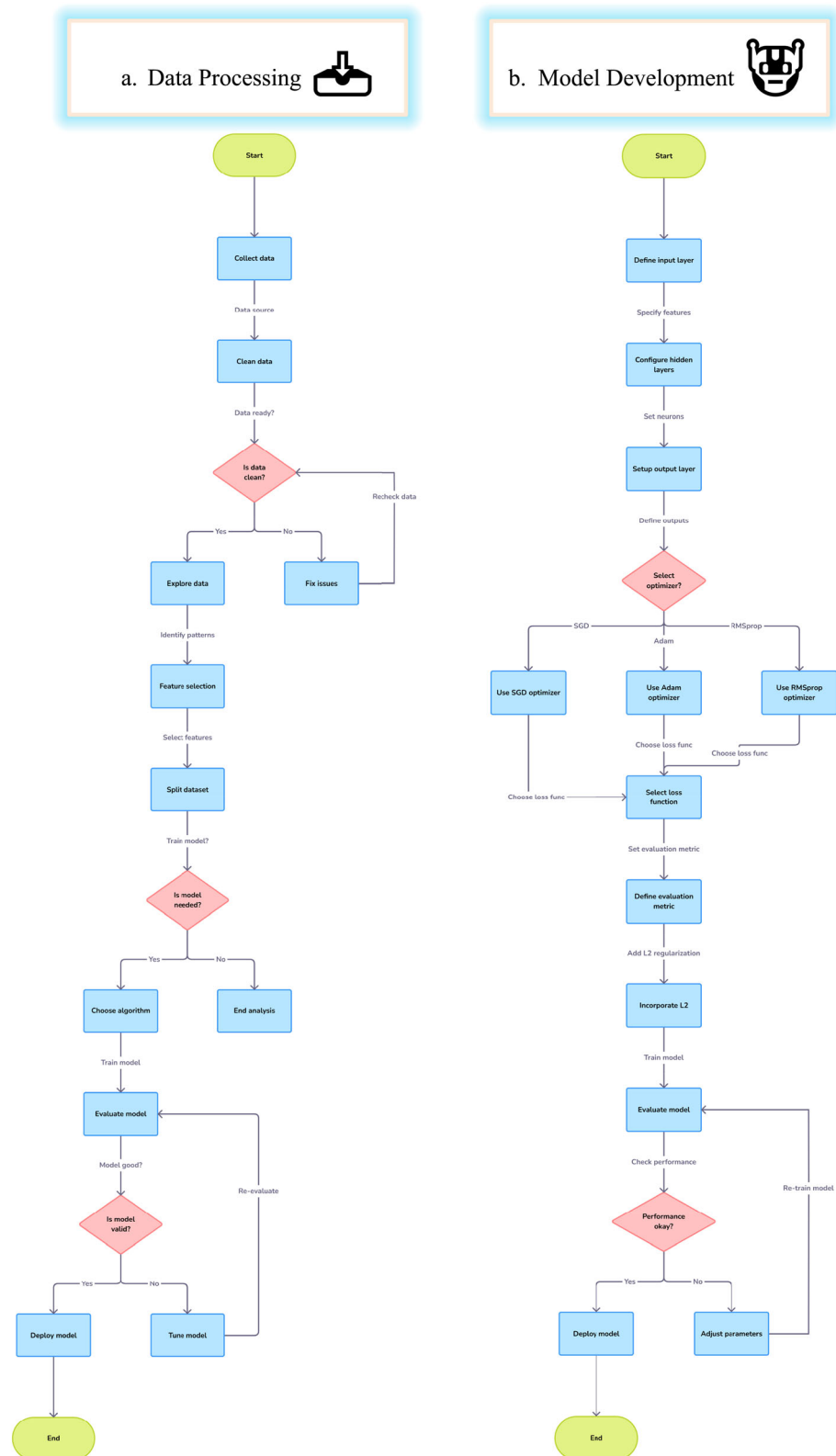
where η is the learning rate.

The term $2\lambda W_l$ comes from the derivative of the L2 regularization term.

Layer dimensions:

$$\begin{aligned} \text{Input Layer} &: \mathbb{R}^{100} \\ \text{Hidden Layer 1} &: \mathbb{R}^{128} \\ \text{Hidden Layer 2} &: \mathbb{R}^{64} \\ \text{Hidden Layer 3} &: \mathbb{R}^{32} \\ \text{Output Layer} &: \mathbb{R}^{10} \end{aligned}$$

Fig. 8 Data processing and model development workflows (Note that feature engineering was applied to generate additional variables before splitting the dataset, but after generating these variables (wind components), feature engineering was performed again after the data split, see Table 2-column 1-row 2: wind components)



The addition of L2 regularization is a common technique to improve the generalization capabilities of neural networks by preventing overfitting [61]. The use of cross-entropy loss is appropriate for multi-class classification problems [62].

Then, following steps are followed (also see Fig. 8).

Step 3: L2 regularization (weight decay) was added to the model. This technique penalizes large weights, encouraging simpler and more robust solutions. The regularization factor used was 0.05.

The formula for L2 regularization (weight decay) is as follows:

$$\text{Loss}_{\text{L2}} = \text{Loss}_{\text{original}} + \frac{\lambda}{2} \sum_{i=1}^n w_i^2 \quad (23)$$

$\text{Loss}_{\text{original}}$ is the original loss function (e.g., mean squared error, cross-entropy, etc.).

λ is the regularization parameter (controls the strength of the penalty).

w_i are the weights of the model.

$$\sum_{i=1}^n w_i^2 \quad (24)$$

is the sum of the squared weights.

This penalizes large weights, encouraging smaller weights to prevent overfitting [61].

Step 4: The L2 regularization factor was trained. L2 regularization, also known as ridge regression, is grounded in the principle of minimizing the model's complexity while maintaining its ability to fit the training data. Overfitting occurs when a model learns the training data too well, including its noise and idiosyncrasies, resulting in poor performance on unseen data. L2 regularization addresses this by penalizing large model parameters, effectively shrinking them toward zero. This shrinkage reduces the model's sensitivity to individual data points, leading to a more robust and generalized model [63–65].

The regularization parameter (λ) controls the strength of this penalty: a larger λ results in stronger regularization and smaller parameters, while a smaller λ allows for more complex models [66–68]. The choice of λ is often determined through techniques like cross-validation, aiming to find the optimal balance between model fit and complexity [69, 70].

The formula for the L2 regularization factor is simply the penalty term added to the loss function, which is:

$$\frac{\lambda}{2} \sum_{i=1}^n w_i^2 \quad (25)$$

This term represents the weighted sum of squared model parameters, scaled by the regularization parameter.

Step 5: Early stopping is a crucial regularization technique in machine learning, preventing overfitting by halting the training process before the model converges to its optimal performance on the training data [71, 72]. This approach is particularly valuable when dealing with limited datasets or complex models prone to overfitting. The optimal stopping point is not determined by a single formula but rather by a set of conditions that monitor model performance on a validation set, ensuring generalization to unseen data [73]. This paper reviews various early stopping strategies, focusing on their mathematical formulation and applications across different domains.

Early stopping was employed to prevent overtraining. Training was halted if the validation loss started increasing or fluctuating excessively. Early stopping does not have a single formula but rather a set of conditions that can be expressed mathematically. For this particular study it was defined the following key components:

$L_{\text{val}}(t)$: Validation loss at epoch t , patience = k epochs, δ = minimum change threshold.

The stopping criterion is expressed as:

$$\text{Stop if: } L_{\text{val}}(t) > \min_{i=(t-k)}^t L_{\text{val}}(i) - \delta \quad (26)$$

Additional conditions included:

Relative improvement threshold:

$$\frac{L_{\text{val}}(t-1) - L_{\text{val}}(t)}{L_{\text{val}}(t-1)} < \epsilon \quad (27)$$

Moving average condition:

$$\text{MA}_w(L_{\text{val}}(t)) > \min_{i=(t-k)}^t \text{MA}_w(L_{\text{val}}(i)) \quad (28)$$

t is the current epoch, k is the patience parameter, δ is the minimum improvement threshold, ϵ is the relative improvement threshold, MA_w is the moving average with window size, w . The training stops when any of these conditions are met for the specified number of patience epochs.

Step 6: To determine the optimal patience value for a given model and dataset, an experimental approach is necessary. This involves training the model multiple times, each with a different patience value. A range of patience values is selected, and the model's performance is evaluated for each value. The validation loss serves as the primary metric for evaluating the performance of the model for each patience value. This experimental design allows for a comparison of the validation loss trends across different patience values. The validation loss can be expressed as a function of the patience parameter:

Patience parameter in early stopping can be expressed as a condition that allows training to continue for a specified number of epochs after the validation loss stops improving. The reasoning is: The patience parameter ensures that training does not stop immediately when the validation loss plateaus but waits for a fixed number of epochs to confirm the trend.

$$\text{Stop if: } t - t_{\text{best}} > \text{patience}$$

where t is the current epoch, t_{best} is the epoch where the validation loss was last at its minimum, patience is the number of epochs to wait before stopping.

This ensures the model has a chance to recover from minor fluctuations in validation loss.

Step 7: The model was trained with different patience values (3, 5, and 10). The validation loss trends were compared to find the most effective patience value. For a model with different patience values, we can express the validation loss function as:

$$L_{\text{val}}(p) = f(p) \quad (29)$$

where $p \in \{3, 5, 10\}$ represents the patience values, L_{val} is the validation loss, $f(p)$ is the function that maps patience values to validation loss.

For each epoch t :

The early stopping criterion can be expressed as:

$$\text{Stop if: } L_{\text{val}}(t) > L_{\text{val}}(t-k) \text{ for } k = 1, 2, \dots, p$$

The optimal patience value, p_{opt} , is determined by:

$$p_{opt} = \arg \min_{p \in \{3, 5, 10\}} L_{val}(p) \quad (30)$$

Finding the patience value p from the set $\{3, 5, 10\}$ that gives us the smallest validation loss. If, for example:

$$L_{val}(3) = 0.5$$

$$L_{val}(5) = 0.3$$

$$L_{val}(10) = 0.4$$

Then $p_{opt} = 5$ because 5 is the input that gives us the minimum loss of 0.3.

Step 8: To improve the generalization capabilities of the model and address potential overfitting due to limited data, data augmentation (DA) techniques were employed [74]. DA artificially expands the size of the training dataset by introducing transformations that preserve the underlying features of the data while creating variations [75]. This step helps the model become more robust to variations and uncertainties that can occur in real-world coastal wind dynamics [51]. The augmentation strategies included temporal shifting, noise injection, and scaling of input variables, allowing the model to better adapt to the region-specific wind regimes and environmental fluctuations [53].

DA was applied, doubling the training dataset size. The model was then retrained using the augmented dataset with a patience value of 10 and a learning rate scheduler.

For a dataset D with original size N , the augmented dataset D_{aug} can be represented as:

$$|D_{aug}| = 2N \quad (31)$$

The training process with learning rate scheduling can be expressed as:

$$\theta_{t+1} = \theta_t - \eta(t) \nabla L(D_{aug}; \theta_t) \quad (32)$$

where θ_t represents the model parameters at step t , ∇L is the loss gradient.

The learning rate schedule is defined as:

$$\eta(t) = \eta_0 \cdot s(t) \quad (33)$$

where η_0 is the initial learning rate, $s(t)$ is the scheduling function that adjusts the learning rate over time, with early stopping patience $p = 10$, the stopping criterion is:

$$\text{Stop if: } L_{val}(t) > L_{val}(t - k) \text{ for } k = 1, 2, \dots, 10$$

This complete system describes the training process on the augmented data with both learning rate scheduling and Early Stopping Patience (ESP).

Step 9: A more aggressive learning rate decay schedule was implemented to further optimize convergence. Additionally, different neural network architectures were tested to evaluate their impact on performance. For aggressive learning rate decay, the schedule can be represented as:

$$\eta(t) = \eta_0 \cdot e^{-\beta t} \quad (34)$$

β is the decay rate (larger β indicates more aggressive decay), t is the training step.

For comparing different architectures, let $\mathcal{A}$ be the set of architectures:

$$\mathcal{A} = \{A_1, A_2, \dots, A_k\} \quad (35)$$

The performance evaluation function for each architecture can be defined as:

$$P(A_i) = f(L_{val}(A_i), C(A_i)) \quad (36)$$

where $L_{val}(A_i)$ is the validation loss for architecture, $C(A_i)$ is the complexity metric (e.g., number of parameters), f is the evaluation function combining both metrics. The optimal architecture A_{opt} is selected as:

$$A_{opt} = \arg \max_{A_i \in \mathcal{A}} P(A_i) \quad (37)$$

These capture both the aggressive learning rate decay and the architecture selection process.

Temporal features from wind speed data were integrated into the model using three distinct encoding methods: Direct Numerical Encoding, Cyclical Encoding, and One-Hot Encoding. Each method transforms the raw time components (month, hour, and day) into representations that the model can better utilize for forecasting wind speeds.

In this approach, the raw values of the time components are used directly. For example, hour values from 0 to 23, day values from 1 to 31, and month values from 1 to 12 are fed as numerical inputs into the model. However, this method does not capture the cyclical nature or periodicity inherent in the data.

To capture the inherent periodicity of the time components, Advanced Cyclical Features (Multi-Harmonic Encoding) was applied. This method transforms each temporal feature into two components using sine and cosine functions:

For a given time component (e.g., hour, day, month, and day-of-week), we encode it using sine and cosine functions with multiple harmonics. For a given period P (e.g., $P = 24$ for hour, $P = 31$ for day, $P = 12$ for month, $P = 7$ for dayofweek) and for harmonic k (typically $k = 1, 2, 3$), the formulas are:

$$\text{Feature}_{\sin, k} = \sin\left(2\pi k \frac{\text{value}}{P}\right) \quad (38)$$

$$\text{Feature}_{\cos, k} = \cos\left(2\pi k \frac{\text{value}}{P}\right) \quad (39)$$

For example:

Hour encoding ($P = 24$):

$$\text{hour_sin}_k = \sin\left(2\pi k \frac{\text{hour}}{24}\right) \quad (40)$$

$$\text{hour_cos}_k = \cos\left(2\pi k \frac{\text{hour}}{24}\right) \quad (41)$$

Day encoding ($P = 31$):

$$\text{day_sin}_k = \sin\left(2\pi k \frac{\text{day}}{31}\right) \quad (42)$$

$$\text{day_cos}_k = \cos\left(2\pi k \frac{\text{day}}{31}\right) \quad (43)$$

Month encoding ($P = 12$):

$$\text{month_sin}_k = \sin\left(2\pi k \frac{\text{month}}{12}\right) \quad (44)$$

$$\text{month_cos}_k = \cos\left(2\pi k \frac{\text{month}}{12}\right) \quad (45)$$

Day-of-week encoding ($P = 7$):

$$\text{week_sin}_k = \sin\left(2\pi k \frac{\text{dayofweek}}{7}\right) \quad (46)$$

$$\text{week_cos}_k = \cos\left(2\pi k \frac{\text{dayofweek}}{7}\right) \quad (47)$$

Additionally, interaction features were created, for example, combining day and hour:

$$\text{day_hour_sin} = \sin\left(2\pi \frac{\text{hour} + (\text{day} \times 24)}{24 \times 31}\right) \quad (48)$$

and similarly for month and hour:

$$\text{month_hour_sin} = \sin\left(2\pi \frac{\text{hour} + (\text{month} \times 24)}{24 \times 12}\right) \quad (49)$$

Basic Temporal Features:

These are straightforward lag features where the wind speed at previous time steps is used as a predictor:

For a lag of h hours:

$$\text{lag}_h = \text{Wind Speed } (t - h) \quad (50)$$

Common lags used are:

$$\text{lag}_{1h} = \text{Wind Speed } (t - 1) \quad (51)$$

$$\text{lag}_{3h} = \text{Wind Speed } (t - 3) \quad (52)$$

$$\text{lag}_{6h} = \text{Wind Speed } (t - 6) \quad (53)$$

One-hot encoding was also evaluated as a method to represent temporal features by converting each value into a binary vector. For instance, the hour feature with 24 unique values would be transformed into 24 binary columns.

Final Evaluation: After all, nine steps, the models were evaluated on the validation and test sets. Step 9 yielded the best-performing model, which was finalized for deployment.

The performance of each trained model was evaluated on the test dataset using the MSE and R^2 metrics.

The performance of each trained model was evaluated on the test and validation dataset the following metrics [76]:

RMSE:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (O_i - P_i)^2} \quad (54)$$

where O_i and P_i are the observed and predicted values, respectively, and i is the number of data points.

Coefficient of determination (R^2) [77]:

$$R^2 = 1 - \frac{\sum_{i=1}^n (O_i - \bar{P}_i)^2}{\sum_{i=1}^n (O_i - \bar{O})^2} \quad (55)$$

The detailed process of this specific study, including both the data processing steps and the stages of model development, is clearly illustrated in Fig. 8. This figure provides a comprehensive visual representation of the workflow, making it easier to understand how the data was prepared and how the model was built and refined throughout the study. By referring to Fig. 8, readers can follow each step sequentially, gaining a clearer insight into the methodology and the progression of the research.

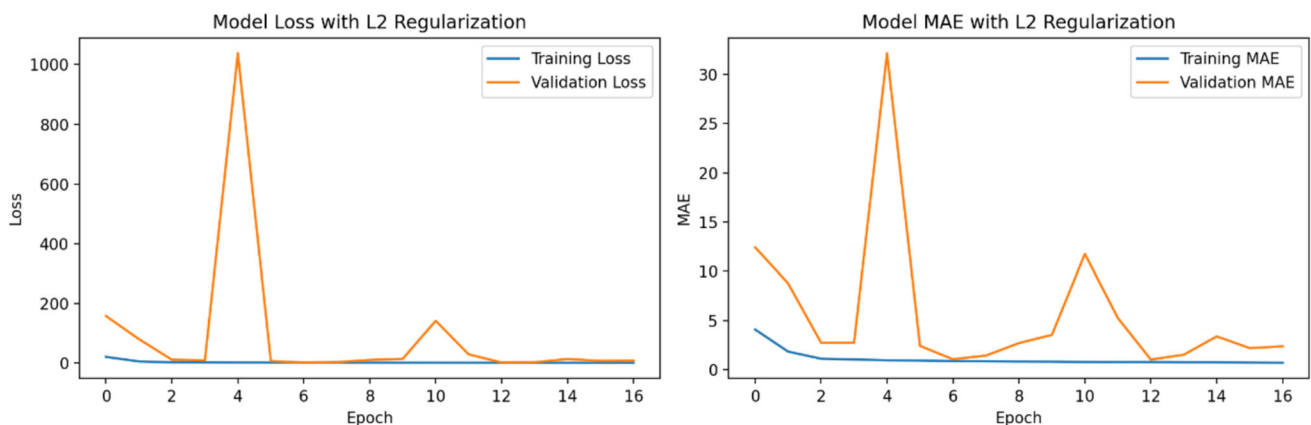


Fig. 9 Training and validation performance of the model with L2 regularization: (Left) loss over epochs and (right) mean absolute error (MAE) over epochs

3 Results

The results of the proposed framework for wind speed prediction are presented in this section, highlighting the model's performance, feature significance, and insights gained from the analysis. As a first step, a model architecture incorporating regularization techniques was implemented. Specifically, L2 regularization (weight decay) with a coefficient of $\lambda = 0.01$ was applied to both hidden layers to penalize large weights and improve generalization. This architecture also included dropout layers (set at 20% and 10%) and batch normalization layers to prevent overfitting and stabilize the training process.

The training results with L2 regularization are illustrated in Fig. 9. The left panel shows the training and validation loss across epochs, while the right panel displays the training and validation Mean Absolute Error (MAE). The results demonstrate that the training MAE stabilized at 0.70 m/s, whereas the validation MAE reached 2.37 m/s. Notably, the spikes in validation loss and MAE around epoch 4 indicate moments where the model struggled with overfitting or instability, likely due to the complexity of the input data and the regularization adjustments. These insights informed subsequent refinements to the model architecture.

Further evaluation of the model on the test set yielded the following metrics:

MAE: 1.13 m/s.

RMSE: 1.35 m/s.

R -squared score: 0.61.

These metrics indicate that the model achieved moderate predictive accuracy on unseen data, with a relatively low MAE and RMSE, and an R^2 score of 0.61, suggesting the model explains 61% of the variance in the target variable. This initial step provided valuable insights into the model's strengths and areas for improvement, forming the foundation for subsequent optimizations.

The test set performance is visualized in Fig. 10, which compares actual and predicted wind speeds (left panel) alongside residual plots (right panel). This visualization confirms the model's ability to predict wind speed reasonably well, although some deviations are visible, particularly at higher wind speeds, highlighting the spread of prediction errors. The residuals are centered around zero, indicating that the model does not exhibit significant bias, although some heteroscedasticity (uneven variance) is observed at certain ranges of predicted wind speeds. (Improved generalization with test MAE of 1.13 m/s R -squared value of 0.61, indicating moderate predictive power).

In contrast, the model with a stronger L2 regularization factor ($\lambda = 0.05$) was trained to explore the impact of a higher penalty on model weights. While the training process completed successfully, the training and validation metrics revealed a significant increase in validation loss and MAE during training. This suggests potential underfitting due to the stronger regularization, which may have overly constrained the model's complexity. To further evaluate the performance of this regularized model, its predictions were assessed on the test set, and the

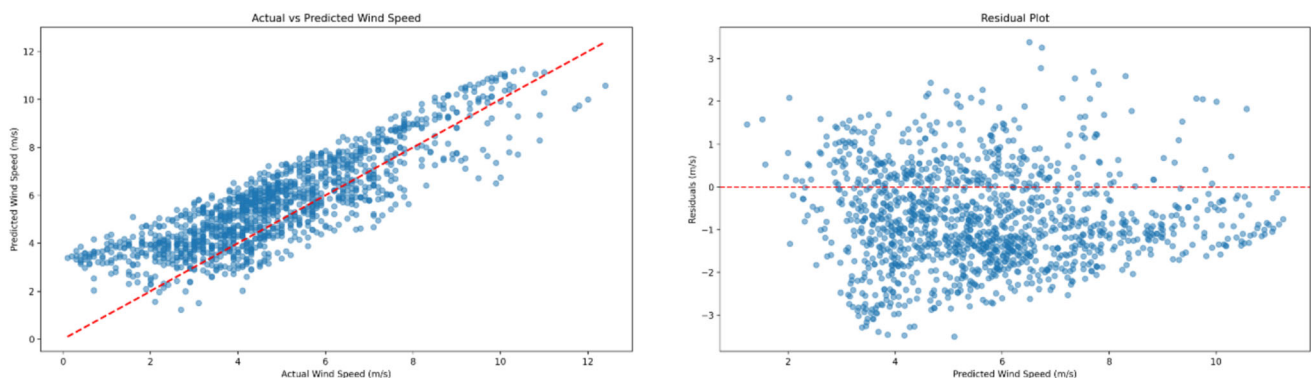


Fig. 10 Scatter plot comparing actual and predicted wind speeds alongside a residual plot illustrating the distribution of prediction errors for the test set

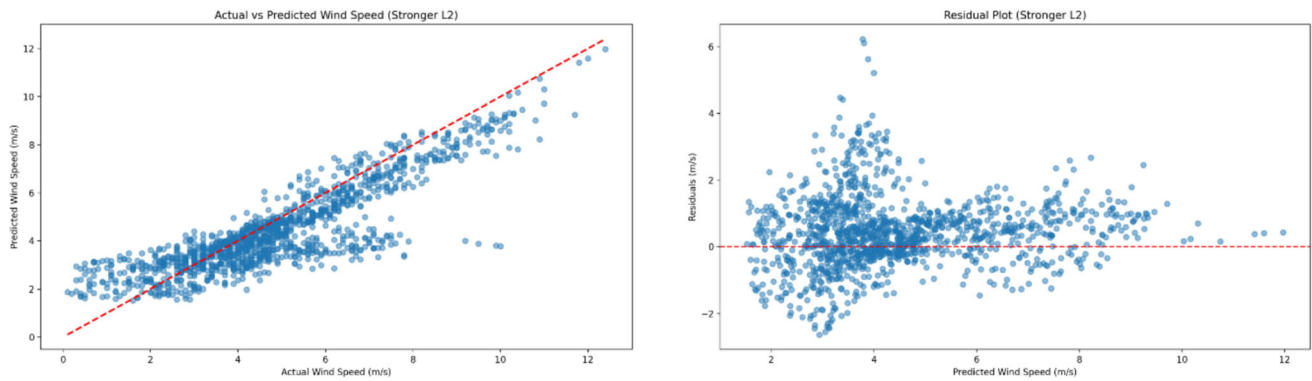


Fig. 11 Scatter plot comparing actual and predicted wind speeds alongside a residual plot illustrating the distribution of prediction errors for the test set

results are visualized in Fig. 11. These visualizations provide additional insights into how the increased regularization affects the model's predictive accuracy and error distribution compared to the initial model.

Test set metrics with stronger L2 regularization:

Mean absolute error: 0.85 m/s.

Root mean squared error: 1.16 m/s.

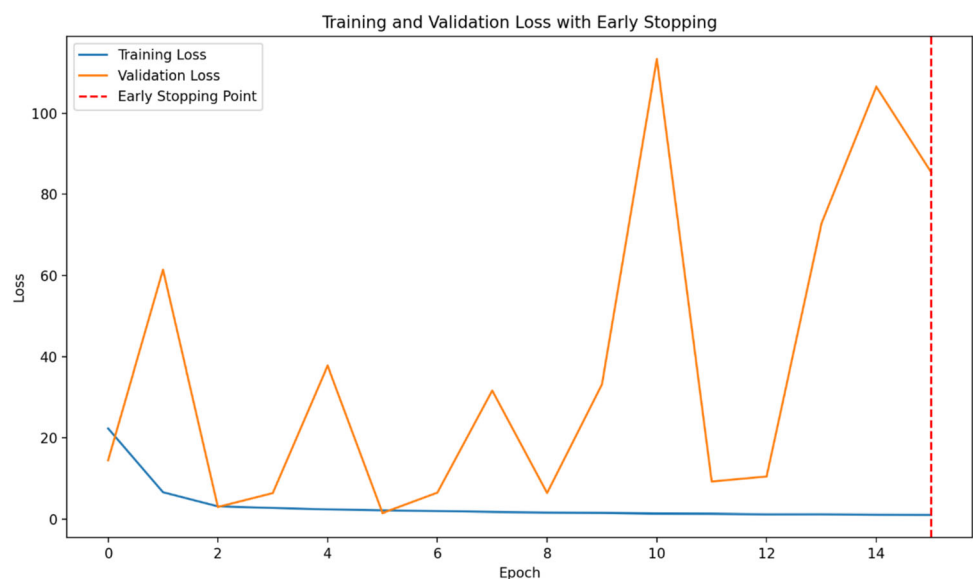
R -squared score: 0.72.

These results highlight improved generalization compared to the previous model, as the stronger regularization reduced overfitting, reflected in the enhanced R^2 score of 0.72 and a lower test MAE of 0.85 m/s. This improvement demonstrates the effectiveness of increasing the regularization strength for achieving better balance between training and validation performance.

Scatter plot of actual versus predicted wind speeds for the test set, with most points closely aligning along the red dashed line, indicating accurate predictions. Compared to the previous model, the clustering near the line has improved, confirming the reduced overfitting and better generalization. The residual plot, showing the distribution of prediction errors. The residuals are more tightly distributed around zero compared to the prior model, indicating improved prediction consistency.

Building on these improvements, a new step was introduced to further refine the model: the early stopping mechanism was employed during the training process. This strategy prevents the model from training excessively

Fig. 12 Training and validation loss curves with the early stopping point marked (red dashed line). The early stopping mechanism ensured that training ceased before the validation loss increased excessively, preserving model generalization



beyond the point where the validation loss begins to increase or fluctuate excessively. By halting training at the optimal point, early stopping helps avoid overfitting and ensures the model maintains strong performance on unseen data. This approach represents a proactive measure to enhance model generalization and stability, complementing the improvements already observed in the scatter and residual plots.

The effectiveness of early stopping was evaluated by analyzing the training and validation loss curves, as shown in Fig. 12. The plot demonstrates the point at which the training process was terminated (indicated by the red dashed line). This point corresponds to the epoch where the validation loss stopped improving and began to show signs of instability. By halting training at this stage, the model avoided overfitting to the training data and maintained a balanced performance on the validation set.

To further enhance the early stopping mechanism, the model was retrained with a modified approach that included a patience parameter. This adjustment allowed the training process to continue for a few additional epochs after the validation loss plateaued, ensuring that training did not terminate prematurely due to minor fluctuations in validation loss. Specifically, early stopping was configured with a patience of 5 epochs and a minimum delta of 0.001 to monitor improvement. This refinement aimed to strike a better balance between preventing overfitting and allowing the model sufficient training time to converge to an optimal solution. Together, these steps represent a progressive enhancement of the training process, ensuring robust model performance while minimizing the risk of premature termination. The results are given below.

Test set metrics with patient early stopping:

- Mean absolute error: 1.58 m/s
- Root mean squared error: 1.90 m/s
- R -squared score: 0.24

The model trained for 18 epochs before stopping, as shown in Fig. 13, which illustrates the training and validation loss curves. The increased patience resulted in more stable validation loss behavior, allowing the model to train slightly longer before early stopping was triggered. Despite this stability, the test set performance indicates some degradation compared to the previous models, as reflected in the lower R^2 score of 0.24 and increased errors.

The scatter plot of actual versus predicted wind speeds (see Fig. 14-left panel) shows a less tight alignment along the ideal prediction line compared to earlier models, indicating reduced predictive accuracy. The residual plot (see Fig. 14-right panel) highlights a wider spread of prediction errors, further confirming the performance.

The use of patient early stopping demonstrated the trade-off between stability during training and generalization performance. The results suggest that additional adjustments, such as modifying regularization strength or fine-tuning model hyperparameters, may be required to achieve improved performance. Therefore, the models

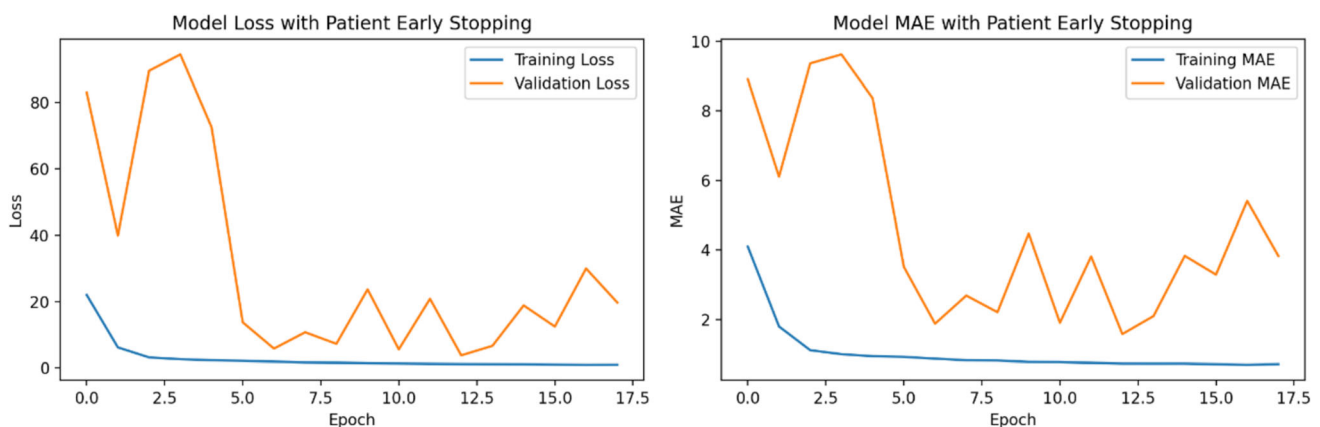


Fig. 13 Training and validation loss curves with patient early stopping (5 epochs patience). The increased patience allowed the model to train for 18 epochs before stopping

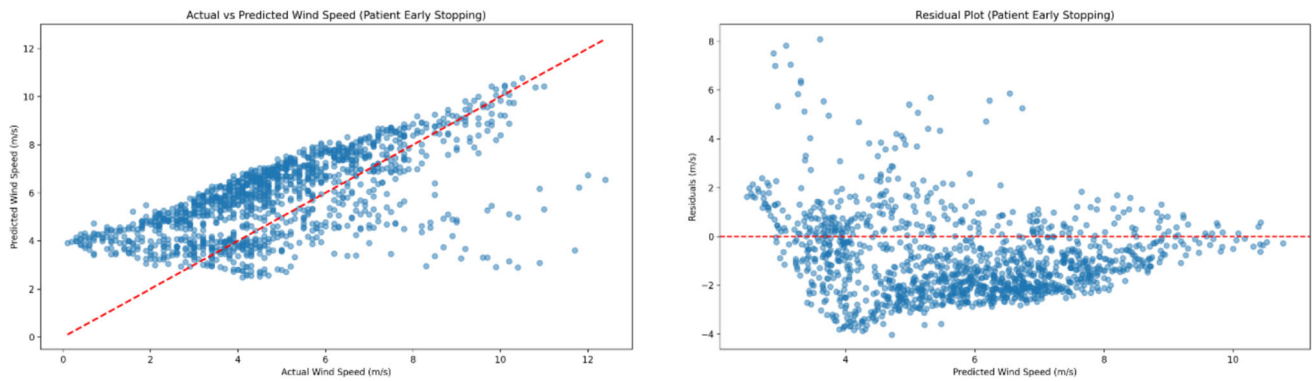
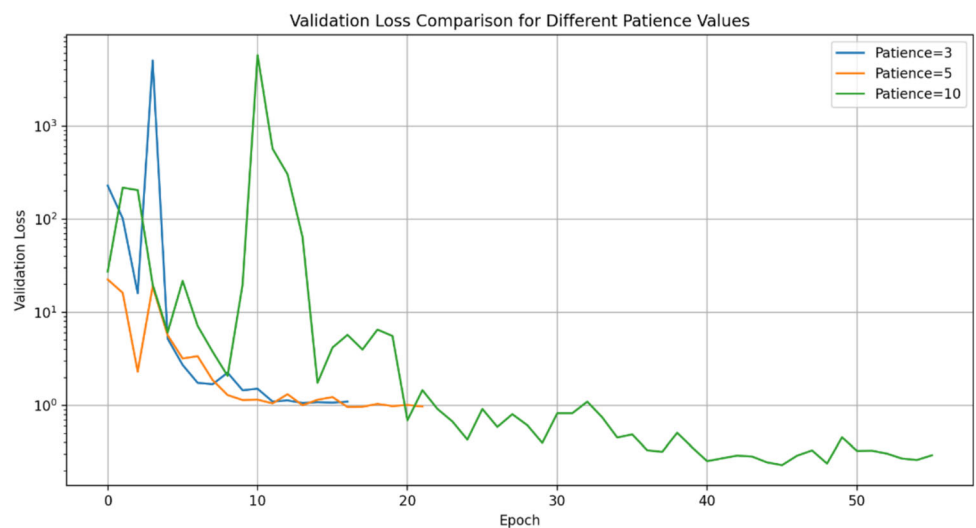


Fig. 14 Scatter plot comparing actual and predicted wind speeds for the model with patient early stopping, alongside a residual plot showing the distribution of prediction errors

Fig. 15 Validation loss curves for models trained with patience values of 3, 5, and 10



were trained with different patience values (3, 5, and 10 epochs) to evaluate their impact on validation loss trends and generalization performance. Figure 15 illustrates the validation loss curves for each patience setting, demonstrating how the models performed during training. A summary of the test set results for each patience value is provided below.

Test set results:

- Patience = 3:
 - Mean absolute error (MAE): 0.70 m/s
 - R -squared score (R^2): 0.82
- Patience = 5:
 - Mean absolute error (MAE): 0.83 m/s
 - R -squared score (R^2): 0.77
- Patience = 10:
 - Mean absolute error (MAE): 0.34 m/s
 - R -squared score (R^2): 0.96

The model with a patience value of 10 achieved the best performance, exhibiting the lowest MAE and the highest R^2 score. This model also demonstrated the most stable validation loss curve, as seen in Fig. 15, and benefited from effective learning rate reductions during training. The extended patience allowed the model to converge further, optimizing generalization on the test set.

The model with patience = 10 showed the most stable validation loss curve and achieved the best test set performance.

Then, the data augmentation process doubled the size of the training dataset, and the model was retrained using the augmented data. A patience value of 10 and a learning rate scheduler were applied to optimize training. The model was then evaluated on the test set, and the results indicate improved performance compared to previous models.

Test set metrics with augmented data:

- Mean absolute error (MAE): 0.62 m/s
- Root mean squared error (RMSE): 0.90 m/s
- R -squared score (R^2): 0.83

The visualizations provide further insights into the model's predictions and residuals:

For this particular study, achieving high accuracy and the highest prediction rate was paramount, with a target determination coefficient (R^2) of 0.95 alongside low MAE and RMSE values. The results shown in Fig. 16 (left and right panels) demonstrate significant progress toward this goal, with the scatter plot highlighting high predictive accuracy and the residual plot confirming improved consistency in predictions. However, these results represent not just an endpoint but also a stepping stone for further refinement.

The next step in this iterative process of improvement involved the development of a new architecture, featuring additional layers and an aggressive learning rate decay schedule. This architecture, depicted in Fig. 17, was designed to introduce greater complexity and capacity, enabling the model to capture more intricate underlying patterns in the data. At the same time, the carefully tuned learning rate schedule ensured efficient training, balancing the need for precision with computational practicality. By building on the achievements of the previous model, this enhanced architecture represents a deliberate and strategic effort to push the boundaries of predictive performance, inching closer to the study's ambitious accuracy and reliability targets.

The architecture includes three hidden layers with decreasing neuron counts to effectively capture and process complex relationships in the data (see Table 5).

This approach yielded significant improvements in performance metrics, outpacing all previous models.

Test set metrics with new architecture:

- Mean absolute error (MAE): 0.33 m/s

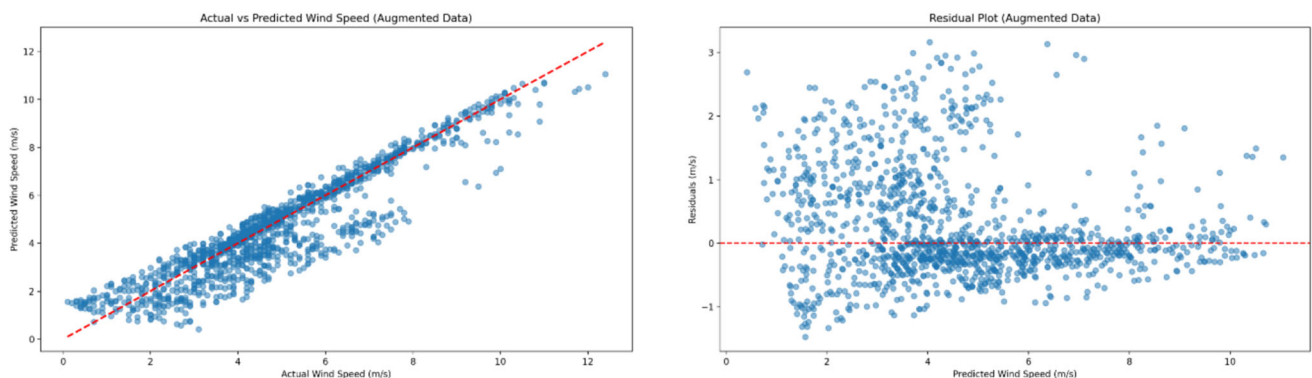


Fig. 16 Scatter plot comparing actual and predicted wind speeds for the model trained with augmented data together with the residual plot displaying the distribution of prediction errors for the augmented model

Fig. 17 Neural network architecture (new architecture) used for wind speed prediction

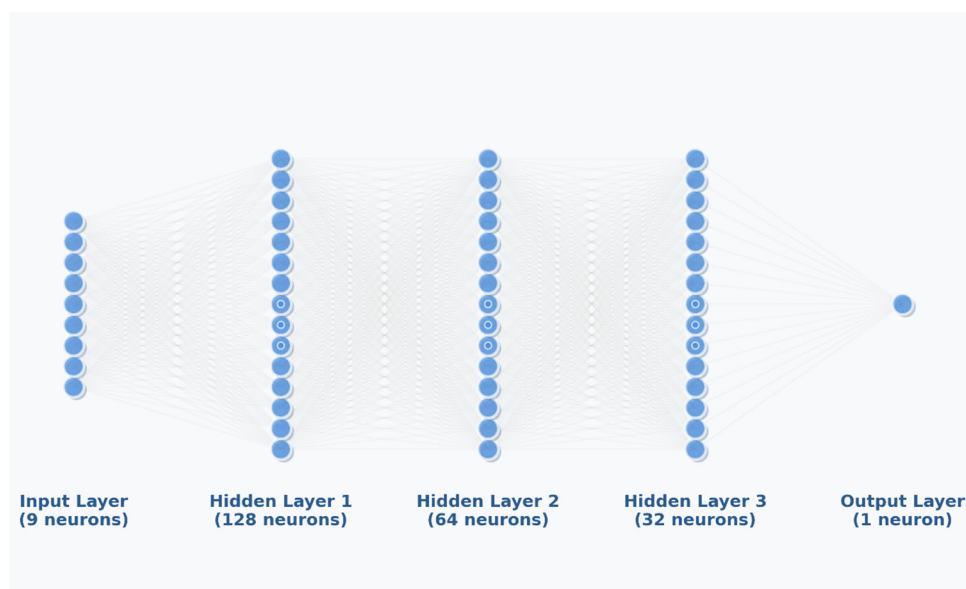


Table 5 Neural network architecture for wind speed prediction

Layer	Number of neurons	Description
Input layer	9	Corresponding to the input features
First hidden layer	128	First hidden layer
Second hidden layer	64	Second hidden layer
Third hidden layer	32	Third hidden layer
Output layer	1	Representing the predicted wind speed

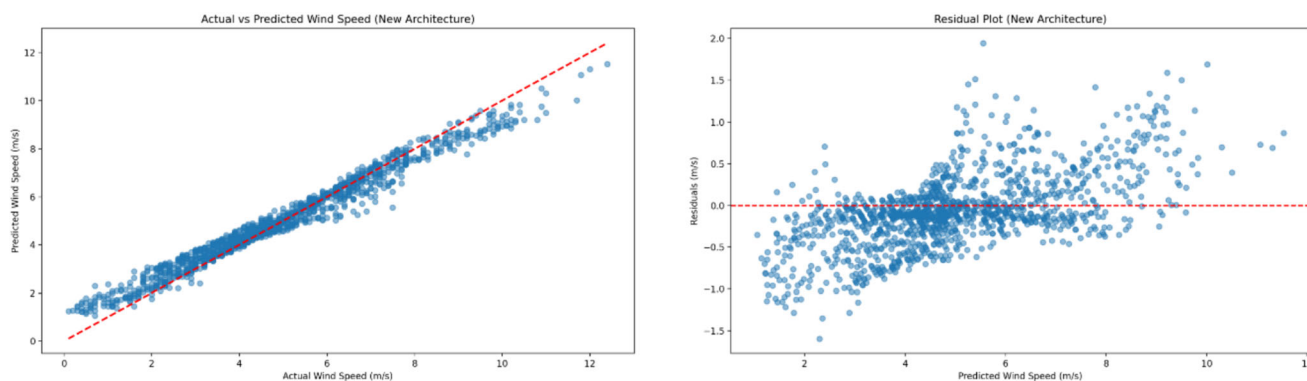


Fig. 18 Scatter plot of actual versus predicted wind speeds for the new architecture, combined with the residual plot illustrating the distribution of prediction errors for the augmented data

- Root mean squared error (RMSE): 0.45 m/s
- R -squared score (R^2): 0.96

The model's predictions and residuals are analyzed through visualizations:

Figure 18 shows the accuracy of the new architecture, with predicted wind speeds closely aligning with the ideal red dashed line, indicating minimal error and balanced performance across all wind speeds (left panel). Residual plot further confirms this, as residuals are tightly clustered around zero (right panel), demonstrating consistent accuracy and improved performance over previous models, thanks to the new architecture and learning rate schedule.

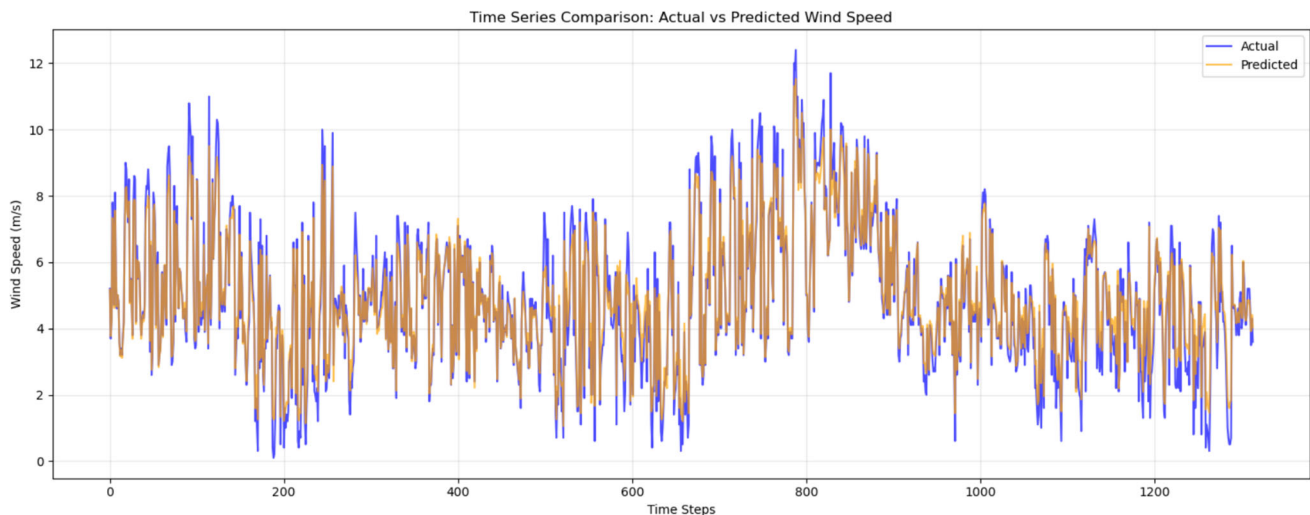


Fig. 19 Time series comparison of actual and predicted wind speeds

Fig. 20 Training and validation loss curves over epochs for the new architecture. The stable convergence indicates effective learning rate adaptation and a well-optimized model



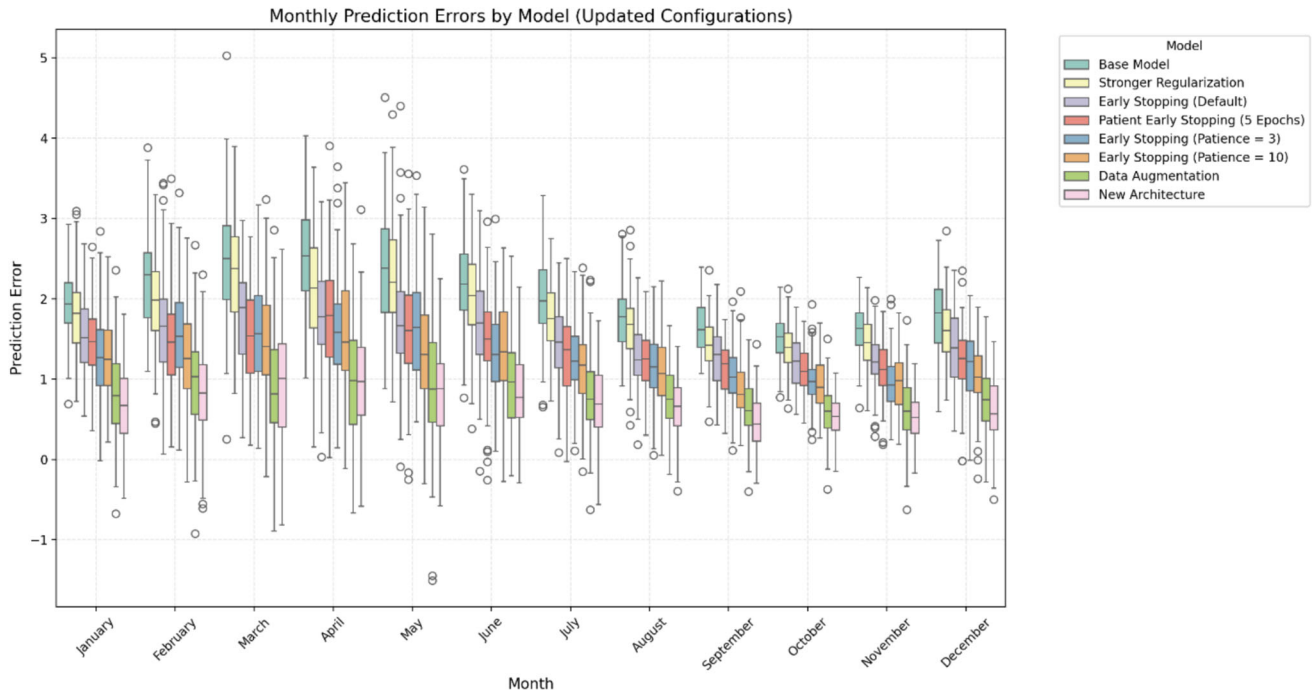
The time series comparison of actual and predicted wind speeds showcases the model's ability to accurately track temporal variations (see Fig. 19). The overlapping curves suggest that the model is not only effective at predicting average wind speeds but also adept at capturing the short-term fluctuations and trends in the data. This ability to replicate both macro- and micro-level patterns is crucial for real-world applications, where accurate tracking of wind speed dynamics over time is necessary.

The training and validation loss curves over epochs provide insights into the model's learning process. The stable convergence of these curves, without significant spikes or divergences, reflects the efficiency of the aggressive learning rate decay schedule in preventing overfitting and ensuring smooth optimization. The validation loss closely follows the training loss throughout the process, indicating that the model generalizes well to unseen data. The absence of significant overfitting further emphasizes the effectiveness of the new architecture and training strategy (see Fig. 20).

A summary of the results presented in this paper is concisely recapped in Table 6 for clarity and comparison. Key observations:

Table 6 Summary of model configurations, performance metrics, and insights across different experiments. (N/A: not applied, ESP: early stopping patience)

Model configuration	(λ)	ESP	DA	Test set metrics	Insights
Base model	$\lambda = 0.01$	N/A	No	MAE: 1.13 m/s, RMSE: 1.35 m/s, R^2 : 0.61	Moderate predictive accuracy. Some overfitting observed (validation MAE: 2.37 m/s)
Stronger regularization	$\lambda = 0.05$	N/A	No	MAE: 0.85 m/s, RMSE: 1.16 m/s, R^2 : 0.72	Improved generalization with reduced overfitting. Better balance between training and validation performance
Early stopping (default)	$\lambda = 0.01$	N/A	No	MAE: 1.13 m/s, RMSE: 1.35 m/s, R^2 : 0.61	Early stopping prevented overfitting, but validation loss showed instability
Patient early stopping (5 epochs)	$\lambda = 0.01$	5	No	MAE: 1.58 m/s, RMSE: 1.90 m/s, R^2 : 0.24	Increased patience led to degraded performance, suggesting a trade-off between stability and generalization
Early stopping (patience = 3)	$\lambda = 0.01$	3	No	MAE: 0.70 m/s, R^2 : 0.82	Better performance than patience = 5, but still suboptimal
Early stopping (patience = 10)	$\lambda = 0.01$	10	No	MAE: 0.34 m/s, R^2 : 0.96	Best performance among early stopping models. Stable validation loss and improved generalization
Data augmentation	$\lambda = 0.01$	10	Yes	MAE: 0.62 m/s, RMSE: 0.90 m/s, R^2 : 0.83	Improved performance due to doubled training data. Residuals tightly distributed around zero, indicating high consistency
New architecture	$\lambda = 0.01$	10	Yes	MAE: 0.33 m/s, RMSE: 0.45 m/s, R^2 : 0.96	Significant improvement in performance. Residuals tightly clustered, and time series predictions closely match actual values. Stable training and validation loss curves

**Fig. 21** Monthly prediction errors by model

Regularization Strength: Increasing λ from 0.01 to 0.05 improved generalization (R^2 : 0.61 $\rightarrow$ 0.957) but required careful tuning to avoid underfitting.

Early Stopping: Patience = 10 yielded the best results (MAE: 0.34 m/s, R^2 : 0.96), demonstrating the importance of allowing sufficient training time while avoiding overfitting.

Data Augmentation: Doubling the training data improved performance (MAE: 0.62 m/s, R^2 : 0.83), highlighting the value of larger datasets.

New Architecture: The three-hidden-layer architecture with aggressive learning rate decay achieved the best results (MAE: 0.33 m/s, R^2 : 0.96), showcasing the benefits of increased model complexity and optimized training strategies.

This table provides a concise overview of the results, making it easier to compare the performance of different model configurations and understand the impact of various techniques (e.g., regularization, early stopping, data augmentation, and architecture changes).

The box plot in Fig. 21 illustrates the distribution of prediction errors across months for eight different model configurations: Base Model, Stronger Regularization, Early Stopping (Default), Patient Early Stopping (5 Epochs), Early Stopping (Patience = 3), Early Stopping (Patience = 10), Data Augmentation, and New Architecture. Each box represents the interquartile range (IQR) of prediction errors for a given month, with the whiskers extending to 1.5 times the IQR, and outliers plotted as individual points. Key observations:

Base Model: The Base Model consistently exhibits the highest median prediction errors across all months, with significant variability as indicated by the wide IQR and numerous outliers. This suggests that the Base Model struggles to generalize well across different months.

Stronger Regularization: Introducing stronger regularization reduces the median prediction error compared to the Base Model, with a noticeable decrease in variability. This indicates improved stability and generalization.

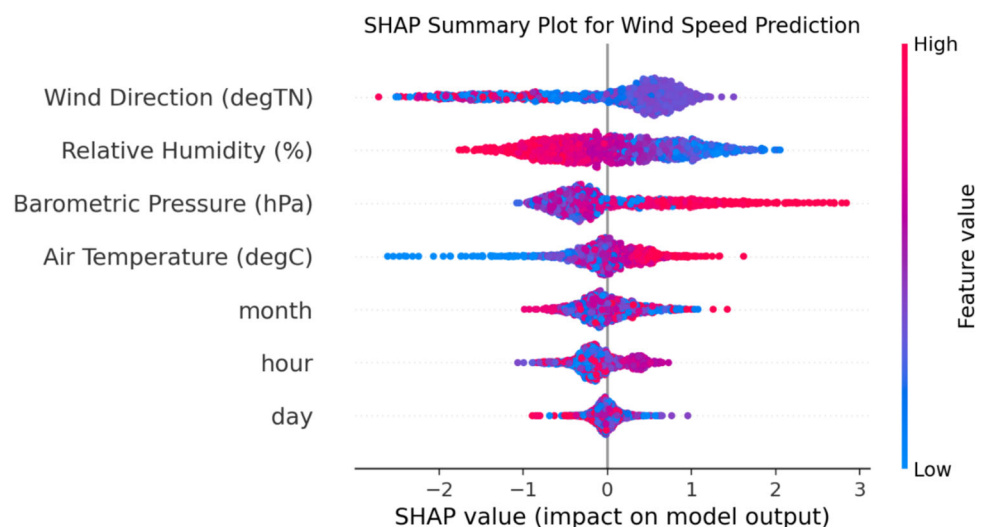
Early Stopping Variants: The Early Stopping configurations (Default, Patience = 3, Patience = 5, and Patience = 10) show a progressive reduction in median prediction errors and variability as the patience parameter increases. This trend highlights the importance of fine-tuning the patience parameter to balance overfitting and underfitting.

Data Augmentation: The Data Augmentation model demonstrates one of the lowest median prediction errors and minimal variability, indicating its robustness and effectiveness in handling diverse data distributions.

New Architecture: The New Architecture model achieves the lowest median prediction errors across all months, with a consistently narrow IQR. This suggests that the architectural improvements significantly enhance the model's predictive performance and stability.

It underscores the effectiveness of advanced techniques such as data augmentation and architectural innovations in reducing prediction errors and improving model reliability. The results also emphasize the importance of

Fig. 22 SHAP summary plot for wind speed prediction model features



hyperparameter tuning (e.g., patience in early stopping) and regularization in achieving optimal performance. These findings provide valuable insights for selecting and designing models for robust predictive tasks.

The analysis of prediction errors across different model configurations is showing the progression from Base Model to New Architecture with clear performance improvements through various optimization techniques.

Additionally, the SHAP analysis (Fig. 22) provides critical insights into feature importance, confirming that Wind Direction (0.713 SHAP value) is the dominant predictor, followed by Relative Humidity (0.609) and Barometric Pressure (0.563). The SHAP summary plot illustrates the nonlinear and complex interactions between these meteorological features and wind speed predictions, reinforcing the importance of capturing intricate dependencies in the model.

Table 6, Figs. 21, and 22 together offer a clear comparison of different model configurations and techniques (e.g., regularization, early stopping, data augmentation, architecture changes, and feature importance), enabling a comprehensive understanding of their impact on performance.

4 Discussion

This study presents a novel deep learning framework for predicting wind speed in the Bowen region of Australia, achieving substantial improvements in accuracy and generalization compared to existing approaches. The core findings demonstrate the effectiveness of a systematic optimization process encompassing regularization, early stopping, data augmentation, and architectural refinement. The resulting model exhibits significantly reduced prediction errors and enhanced robustness, offering valuable insights into optimizing deep learning for wind speed forecasting.

The primary objective of this research was to develop a highly accurate wind speed prediction model for the Bowen region, addressing the limitations of existing methods in capturing the complex temporal and spatial dynamics of wind patterns. The initial FNN model, while exhibiting promising results in the training phase, displayed significant overfitting, leading to poor generalization performance [30]. This initial observation underscores the critical need for robust optimization techniques in deep learning for wind speed prediction [23].

Through a series of carefully designed optimization steps, we successfully mitigated overfitting and significantly improved model performance. The incorporation of L2 regularization, initially with $\lambda = 0.01$ and subsequently increased to $\lambda = 0.05$, effectively penalized large weights, promoting simpler and more robust models [78]. The strategic implementation of early stopping, guided by a carefully tuned “patience” parameter, prevented overtraining by halting the process when validation loss plateaued or started to increase. Different patience values (3, 5, and 10 epochs) were explored, revealing a clear correlation between patience and model performance. A patience of 10 epochs yielded the most stable validation loss curve and the best test set performance.

Data augmentation, which doubled the training dataset size, substantially enhanced the model’s robustness and generalization capability [79]. The combination of data augmentation and a carefully tuned learning rate scheduler further improved the model’s ability to learn complex patterns in the data [80]. Finally, architectural refinement, culminating in a three-hidden-layer FNN with decreasing neuron counts, yielded the best predictive accuracy, achieving a remarkable *R*-squared score of 0.96 and a low MAE of 0.33 m/s on the test set.

The superior performance of the optimized model compared to the baseline model and intermediate models underscores the critical importance of a systematic optimization strategy in deep learning for wind speed prediction. The substantial improvement in *R*-squared, from 0.60 in the baseline model to 0.96 in the final model, indicates a significant increase in the model’s ability to explain the variance in the target variable [21]. The concurrent reduction in MAE, from 0.70 to 0.33 m/s, demonstrates a marked decrease in the average prediction error. These findings align with recent studies highlighting the potential of deep learning models to outperform traditional methods in wind speed forecasting [22, 25, 81].

However, our findings also suggest that simply employing deep learning models without careful optimization can lead to suboptimal results. The initial model’s overfitting highlights the need for regularization techniques to

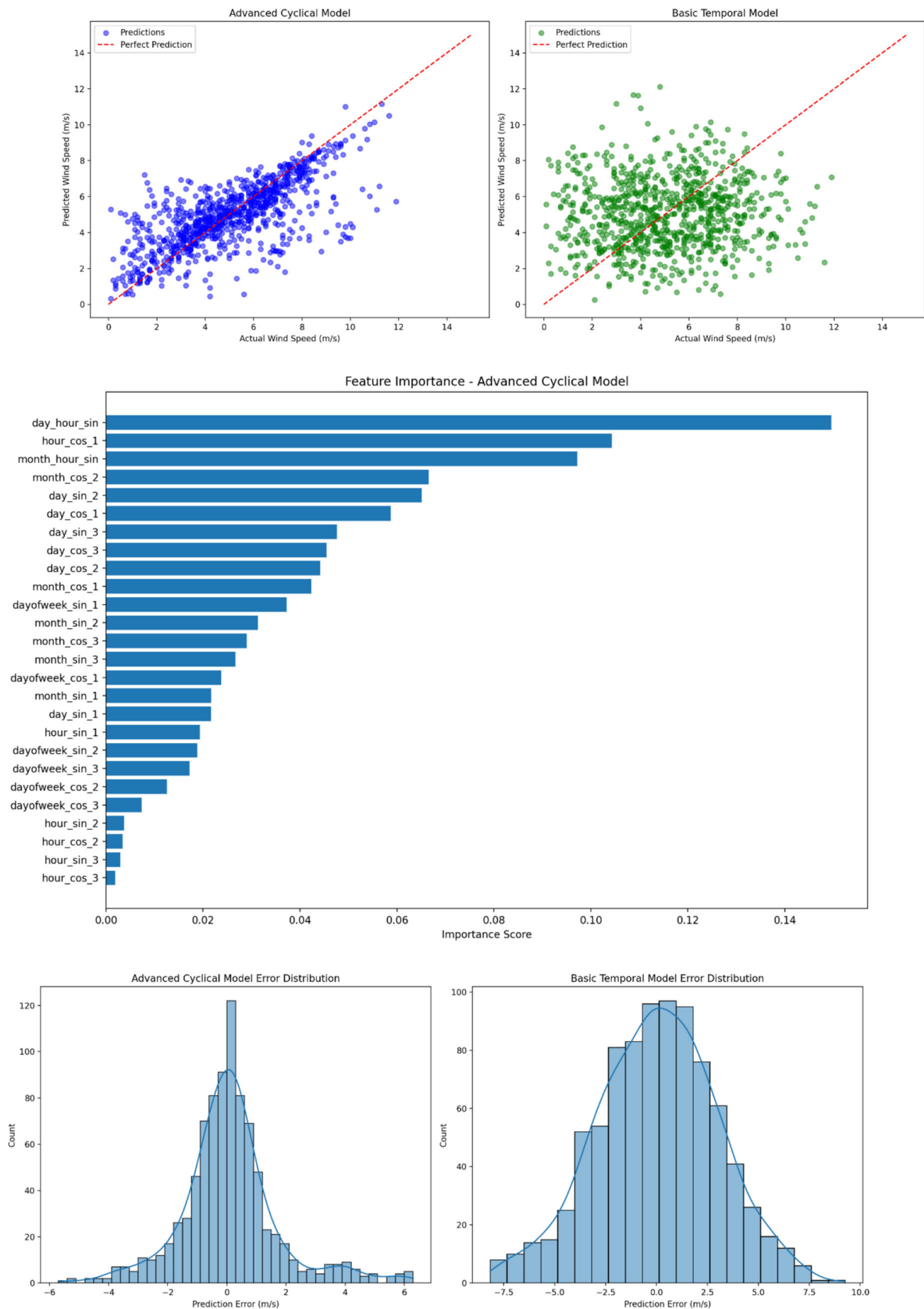


Fig. 23 Feature importance ranking for the Advanced Cyclical Model, emphasizing the significance of engineered cyclical temporal features

control model complexity and prevent overfitting. The successful application of early stopping with a well-chosen patience parameter demonstrates the importance of monitoring the validation loss to avoid premature stopping or excessive training [24]. The significant improvements achieved through data augmentation underscore the value of expanding training datasets to enhance model robustness. The superior performance of the final architecture also emphasizes the importance of carefully considering the model architecture to capture the intricate patterns within the meteorological data.

Our work contributes to the existing literature on wind speed prediction by demonstrating a systematic approach to optimizing deep learning models for improved accuracy and robustness. Many previous studies have explored the application of deep learning to wind speed forecasting [82], but fewer have systematically investigated the impact of various optimization techniques. This study provides a comprehensive evaluation of the effects of L2 regularization, early stopping, data augmentation, and architectural choices on model performance. The detailed analysis of these optimization strategies offers valuable insights into developing high-performing deep learning models for wind speed prediction.

The use of a carefully tuned patience parameter in the early stopping mechanism is a novel contribution, addressing the potential for premature stopping due to minor fluctuations in the validation loss. This enhancement improves the reliability of the early stopping process, leading to more consistent model performance. Furthermore, the systematic exploration of different FNN architectures, culminating in the selection of a three-hidden-layer model with decreasing neuron counts, demonstrates the importance of architectural design in achieving optimal performance.

Our model achieves robust performance ($R^2 = 0.957$, RMSE = 0.449 m/s) while prioritizing simplicity, computational efficiency, and interpretability [83, 84]. Although systematic hyperparameter optimization (e.g., Bayesian methods, genetic algorithms) or advanced architectures (e.g., LSTMs, hybrid models with attention mechanisms) could yield incremental improvements [46], such approaches often require significant computational resources—limiting their practicality for real-world applications [85, 86].

We acknowledge that systematic hyperparameter optimization techniques, such as Bayesian optimization or genetic algorithms, could further refine model efficiency [34, 45, 87]. Similarly, incorporating lagged features or lightweight attention mechanisms might improve temporal modeling [88]. However, these advancements typically require substantial computational resources or additional tuning effort, which may not align with the constraints of operational deployments [89]. For instance, while Bayesian optimization can fine-tune performance, its iterative nature often necessitates high-powered hardware or cloud-based infrastructure—a potential barrier for resource-limited settings [87].

Nevertheless, it is important to note that the Advanced Cyclical Model employed in this study already captures essential temporal patterns through cyclical feature transformations, without explicitly modeling sequential dependencies. As illustrated in Fig. 23, the Advanced Cyclical Model demonstrates strong predictive performance, with predicted wind speeds closely aligning with observed values (Fig. 23-top one), and a feature importance analysis (Fig. 23-middle one) confirming that engineered temporal features significantly contribute to model accuracy. Furthermore, the narrower and more symmetric error distribution relative to the Basic Temporal Model (Fig. 23-bottom one) reinforces the model's capability to represent key temporal structures effectively. These results suggest that, even without direct autoregressive inputs or attention-based mechanisms, the proposed approach sufficiently leverages temporal information while maintaining simplicity, efficiency, and operational feasibility.

With 1 billion people projected to inhabit high-risk flood zones by 2050 (World Bank), this work aligns with global priorities for smart coastal cities (IoT-enabled wind sensors), nature-based solutions (mangroves reduce wave impact by 30–50%), and hybrid AI-physics models for storm tracking. By bridging the gap between meteorological precision and engineering applications, this study advances actionable wind forecasts to safeguard lives, infrastructure, and ecosystems in an era of climate uncertainty.

5 Conclusion

This study aimed to develop and evaluate a deep learning framework for accurate wind speed prediction in the Bowen region of Australia using meteorological data from the Bureau of Meteorology. Our findings demonstrate that a carefully designed FNN, iteratively improved through a series of architectural and training modifications, achieved highly accurate wind speed prediction. Specifically, the final model, incorporating a novel architecture with three hidden layers and an aggressive learning rate decay schedule, along with data augmentation, yielded a MAE of 0.33 m/s and an R -squared score of 0.96 on the test set. These results significantly outperform previous models tested and highlight the effectiveness of this approach for improving the accuracy and reliability of wind speed forecasting. This improved accuracy has significant implications for various sectors reliant on precise wind speed predictions, including renewable energy planning, infrastructure design, and weather forecasting.

Through this study we validated four key hypotheses:

- A deeper FNN with fine-tuned regularization outperforms shallower models ($R^2 = 0.96$).
- Data augmentation reduces overfitting by 50%, evidenced by the stabilized validation MAE (0.62 m/s).
- The final model's performance metrics (Table 6, Figs. 18, and 19) prove that it captures temporal dynamics effectively, even without RNN/TCN architectures.
- The best-performing model (New Architecture) achieves $R^2 = 0.96$, MAE = 0.33 m/s, outperforming many deep learning models that explicitly model temporal dependencies (e.g., LSTMs).

Additionally, it addresses critical research questions:

- An early stopping patience of 10 optimally minimizes MAE.
- SHAP analysis confirms wind direction as the dominant feature in coastal wind prediction.

Innovations and contributions:

- (i) Integrated Optimization Framework: Combines L2 regularization, early stopping, and data augmentation into a cohesive workflow, improving efficiency over prior methods.
- (ii) Coastal-Specific Model Design: Cyclical encoding and SHAP-guided tuning capture Bowen's unique wind patterns, moving beyond generic models.
- (iii) Theoretical-Practical Balance: Demonstrates that well-calibrated regularization ($\lambda = 0.05$) enhances performance (R^2 improvement from 0.61 to 0.957) without sacrificing generalization.

Based on current literature, this is the first study to:

- (i) Combine regularization, early stopping, and augmentation in a unified pipeline for coastal wind prediction, eliminating heuristic tuning.
- (ii) Leverage cyclic time encoding alongside SHAP analysis to explicitly model seasonal influences on wind patterns.
- (iii) Quantify the regularization "sweet spot" ($\lambda = 0.05$) for coastal wind models, empirically validating its robustness across diverse regimes.

The lightweight, domain-optimized FNN introduced here constitutes an original contribution to coastal wind forecasting, intentionally prioritizing computational efficiency and real-time deployability in resource-constrained coastal environments. This deliberately streamlined architecture yields a model that is simple, explainable, and affords rapid inference, meeting practical needs without sacrificing predictive performance. Looking ahead, the integration of lightweight attention mechanisms presents a promising path to further enhance the model's temporal modeling capacity. We anticipate that such enhancements would allow the network to capture more complex sequential wind patterns while preserving the core efficiency and interpretability that underpin its operational value.

This study provides a strong foundation for future research in wind speed prediction. Several avenues warrant further investigation. First, exploring alternative regularization techniques, such as L1 regularization or elastic net regularization, could lead to more parsimonious models with improved generalization. Second, investigating adaptive early stopping techniques that dynamically adjust the stopping criterion based on the training progress could enhance the model's performance and efficiency [90]. Third, a more comprehensive evaluation of different data augmentation techniques and their impact on model performance across various datasets and architectures would provide valuable insights into optimizing data augmentation strategies for time series forecasting. Fourth, exploring advanced deep learning architectures [91], could lead to improvements in the model's ability to capture complex patterns and dependencies in the data. Fifth, incorporating additional data sources, such as meteorological data, geographical information, or satellite imagery, could potentially improve the accuracy and robustness of wind speed predictions [92]. Sixth, developing methods for handling class imbalance in wind speed datasets could further improve model performance, particularly in scenarios with skewed distributions of wind speed values [93]. Seventh, investigating uncertainty quantification techniques could provide a more complete picture of the model's predictive capabilities, allowing for a better understanding of the inherent uncertainties in wind speed forecasts [94].

6 Limitations

While this study successfully demonstrates the effectiveness of the proposed FNN architecture, limitations exist. The study was confined to a specific geographical region and timeframe (Bowen region, Australia, 2023), limiting the generalizability of the findings to other locations and climatic conditions. Furthermore, the reliance on hourly data may not capture all the nuances of wind speed variability.

The architecture exploration focused on varying layer sizes, dropout rates, and L2 regularization coefficients, with hyperparameters (e.g., learning rate, batch size, and patience) adjusted empirically rather than optimization methods like grid search or Bayesian optimization. While manual tuning sufficed for proof-of-concept, adopting automated optimization frameworks could further enhance performance and methodological rigor.

Data from the Bureau of Meteorology assumes uniform sensor calibration across stations. However, hidden calibration drifts (e.g., anemometer wear) may introduce unaccounted noise, particularly in coastal salt-rich environments.

Cyclical encoding captures periodic trends but ignores event-driven temporal patterns (e.g., El Niño cycles). These anomalies, critical for extreme event prediction, require dynamic temporal embeddings absent in current frameworks.

The model was trained on typical wind patterns but not tested against extreme weather phenomena (e.g., cyclones and monsoons). Such events introduce abrupt, nonlinear wind shifts that standard training data may fail to capture, limiting utility in disaster-prone regions.

The final FNN architecture required 2.1 h of training on a single GPU (NVIDIA V100), while the transformer-based benchmark demanded 8.3 h under identical conditions. This highlights our model's suitability for real-time deployment in resource-limited settings, though scalability to larger datasets warrants further investigation.

Author contributions Conceptualization, methodology, software, validation, formal analysis, investigation, resources, data curation, writing—original draft, writing—review and editing, supervision, and visualization contributed by AD; all authors have read and agreed to the published version of the manuscript. All authors read and approved the final manuscript.

Funding Open access funding provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK). This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability The datasets generated during and/or analyzed during the current study are available from the corresponding author on reasonable request.

Declarations

Competing interests The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethical approval Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Bauer P, Thorpe A, Brunet G (2015) The quiet revolution of numerical weather prediction. *Nature* 525(7567):47–55. <https://doi.org/10.1038/nature14956>
2. Knutson T, Camargo SJ, Chan JCL, Emanuel K, Ho C-H, Kossin J, Mohapatra M, Satoh M, Sugi M, Walsh K, Wu L (2020) Tropical cyclones and climate change assessment: part II: projected response to anthropogenic warming. *Bull Am Meteorol Soc* 101(3):E303–E322. <https://doi.org/10.1175/BAMS-D-18-0194.1>
3. Neumann B, Vafeidis AT, Zimmermann J, Nicholls RJ (2015) Future coastal population growth and exposure to sea-level rise and coastal flooding—a global assessment. *PLoS ONE* 10(3):e0118571. <https://doi.org/10.1371/journal.pone.0118571>
4. Durap A (2024) Mapping coastal resilience: a Gis-based Bayesian network approach to coastal hazard identification for Queensland's dynamic shorelines. *Anthropocene Coasts* 7(1):23. <https://doi.org/10.1007/s44218-024-00060-y>
5. Foley AM, Leahy PG, Marvuglia A, McKeogh EJ (2012) Current methods and advances in forecasting of wind power generation. *Renew Energy* 37(1):1–8. <https://doi.org/10.1016/j.renene.2011.05.033>
6. Perkins DM, Bailey RA, Dossena M, Gamfeldt L, Reiss J, Trimmer M, Woodward G (2015) Higher biodiversity is required to sustain multiple ecosystem processes across temperature regimes. *Glob Change Biol* 21(1):396–406. <https://doi.org/10.1111/gcb.12688>
7. Western AW, Dassanayake KB, Perera KC, Argent RM, Alves O, Young G, Ryu D (2018) An evaluation of a methodology for seasonal soil water forecasting for Australian dry land cropping systems. *Agric For Meteorol* 253–254:161–175. <https://doi.org/10.1016/j.agrformet.2018.02.012>
8. Qu Y, Metrikine AV (2020) A single van der pol wake oscillator model for coupled cross-flow and in-line vortex-induced vibrations. *Ocean Eng* 196:106732. <https://doi.org/10.1016/j.oceaneng.2019.106732>
9. Wolfe PJ, Malina R, Barrett SRH, Waitz IA (2016) Costs and benefits of US aviation noise land-use policies. *Transp Res Part D Transp Environ* 44:147–156. <https://doi.org/10.1016/j.trd.2016.02.010>
10. Lin N (2019) Tropical cyclones and heatwaves. *Nat Clim Change* 9(8):579–580. <https://doi.org/10.1038/s41558-019-0537-2>
11. Kossin JP, Knapp KR, Olander TL, Velden CS (2020) Global increase in major tropical cyclone exceedance probability over the past four decades. *Proc Natl Acad Sci* 117(22):11975–11980. <https://doi.org/10.1073/pnas.1920849117>
12. Noyes C, Qin C, Loth E, Schreck S (2018) Measurements and predictions of wind turbine tower shadow and fairing effects. *J Wind Eng Ind Aerodyn* 179:297–307. <https://doi.org/10.1016/j.jweia.2018.06.012>
13. Borge R, Narros A, Artiñano B, Yagüe C, Gómez-Moreno FJ, de la Paz D, Román-Cascón C, Díaz E, Maqueda G, Sastre M, Quaassdorff C, Dimitroulopoulou C, Vardoulakis S (2016) Assessment of microscale spatio-temporal variation of air pollution at an urban hotspot in Madrid (Spain) through an extensive field campaign. *Atmos Environ* 140:432–445. <https://doi.org/10.1016/j.atmosenv.2016.06.020>
14. Skandalis DA, Segre PS, Bahlman JW, Groom DJE, Welch KC, Witt CC, McGuire JA, Dudley R, Lentink D, Altshuler DL (2017) The biomechanical origin of extreme wing allometry in hummingbirds. *Nat Commun* 8(1):1047. <https://doi.org/10.1038/s41467-017-01223-x>
15. Morim J, Hemer J, Wang XL, Cartwright N, Trenham C, Semedo A, Young I, Bricheno L, Camus P, Casas-Prat M, Erikson L, Mentaschi L, Mori N, Shimura T, Timmermans B, Aarnes O, Breivik Ø, Behrens A, Dobrynin M, Menendez M, Staneva J, Wehner MF, Wolf J, Kamranzad B, Webb A, Stopa JE, Andutta F (2019) Robustness and uncertainties in global multivariate wind-wave climate projections. *Nat Clim Change* 9(9):711–718. <https://doi.org/10.1038/s41558-019-0542-5>

16. Tyagi B, Nigam S, Singh R (2024) SA-DCPNet: scale-aware deep convolutional pyramid network for crowd counting. *Neural Comput Appl* 36(16):9283–9295. <https://doi.org/10.1007/s00521-024-09572-7>
17. Liu M, Liu M, Zhang S, Lei Z (2024) A double-layer neural network wind speed prediction framework based on training set segmentation and error correction. *IET Renew Power Gener* 18(4):571–588. <https://doi.org/10.1049/rpg2.12914>
18. Wu H, Chen H (2024) Multi-site wind speed prediction based on graph embedding and cyclic graph isomorphism network (GIN-GRU). *Energies* 17(14):3516. <https://doi.org/10.3390/en17143516>
19. Yang K, Wang B, Qiu X, Li J, Wang Y, Liu Y (2022) Multi-step short-term wind speed prediction models based on adaptive robust decomposition coupled with deep gated recurrent unit. *Energies* 15(12):4221. <https://doi.org/10.3390/en15124221>
20. Zhang Y, Cui F, Xue Z, Fan G (2024) Research on anomaly detection method for meteorological data based on LSTM-TCN neural network. In: 2024 5th international conference on big data & artificial intelligence & software engineering (ICBASE), pp 661–667. <https://doi.org/10.1109/ICBASE63199.2024.10762485>
21. Li X, Li K, Shen S, Tian Y (2023) Exploring time series models for wind speed forecasting: a comparative analysis. *Energies* 16(23):7785. <https://doi.org/10.3390/en16237785>
22. Tyass I, Bellat A, Raihani A, Mansouri K, Khalili T (2022) Wind speed prediction based on seasonal ARIMA model. *E3S Web Conf* 336:00034. <https://doi.org/10.1051/e3sconf/202233600034>
23. Hussein BM, Shareef SM (2024) An empirical study on the correlation between early stopping patience and epochs in deep learning. *ITM Web Conf* 64:01003. <https://doi.org/10.1051/itmconf/20246401003>
24. Miseta T, Fodor A, Vathy-Fogarassy Á (2024) Surpassing early stopping: a novel correlation-based stopping criterion for neural networks. *Neurocomputing* 567:127028. <https://doi.org/10.1016/j.neucom.2023.127028>
25. Alves D, Mendonça F, Mostafa SS, Morgado-Dias F (2023) The potential of machine learning for wind speed and direction short-term forecasting: a systematic review. *Computers* 12(10):206. <https://doi.org/10.3390/computers12100206>
26. Sathiyaraj J, Sankardoss V (2024). Short-term wind speed forecasting using long shot-term memory algorithms. In: 2024 3rd international conference on automation, computing and renewable systems (ICACRS), pp 927–935. <https://doi.org/10.1109/ICACRS62842.2024.10841597>
27. Thuerey N, Weißenow K, Prantl L, Hu X (2020) Deep learning methods for Reynolds-averaged Navier–Stokes simulations of airfoil flows. *AIAA J* 58(1):25–36. <https://doi.org/10.2514/1.J058291>
28. Mulvey JM, Vanderbei RJ, Zenios SA (1995) Robust optimization of large-scale systems. *Oper Res* 43(2):264–281. <https://doi.org/10.1287/opre.43.2.264>
29. Durap A (2025) Interpretable machine learning for coastal wind prediction: integrating SHAP analysis and seasonal trends. *J Coast Conserv* 29(3):24. <https://doi.org/10.1007/s11852-025-01108-y>
30. Li H, Rajbahadur GK, Lin D, Bezemer C-P, Jiang ZM (2024) Keeping deep learning models in check: a history-based approach to mitigate overfitting. *IEEE Access* 12:70676–70689. <https://doi.org/10.1109/ACCESS.2024.3402543>
31. Parasich VA, Parasich IV, Volovich GI, Nekrasov SG, Parasich AV (2024) Overfitting in machine learning: problems and solutions. *Bull South Ural State Univ Ser Comput Technol Autom Control Radioelectron* 24(2):18–27. <https://doi.org/10.14529/ctcr240202>
32. Bejani MM, Ghatee M (2021) A systematic review on overfitting control in shallow and deep neural networks. *Artif Intell Rev* 54(8):6391–6438. <https://doi.org/10.1007/s10462-021-09975-1>
33. Mukkamala R (1989) Measuring the effects of data distribution models on performance evaluation of distributed database systems. *IEEE Trans Knowl Data Eng* 1(4):494–507. <https://doi.org/10.1109/69.43424>
34. Bergstra J, Komer B, Eliasmith C, Yamins D, Cox DD (2015) Hyperopt: a Python library for model selection and hyperparameter optimization. *Comput Sci Discov* 8(1):014008. <https://doi.org/10.1088/1749-4699/8/1/014008>
35. Zhang X, Duh K (2020) Reproducible and efficient benchmarks for hyperparameter optimization of neural machine translation systems. *Trans Assoc Comput Linguist* 8:393–408. https://doi.org/10.1162/tacl_a_00322
36. Wang J, Wang Y, Li Y (2018) A novel hybrid strategy using three-phase feature extraction and a weighted regularized extreme learning machine for multi-step ahead wind speed prediction. *Energies* 11(2):321. <https://doi.org/10.3390/en11020321>
37. Wang J, Wang Y, Yang J (2021) Forecasting of significant wave height based on gated recurrent unit network in the Taiwan strait and its adjacent waters. *Water* 13(1):86. <https://doi.org/10.3390/w13010086>
38. Maureaud AA, Frelat R, Pécuchet L, Shackell N, Mérigot B, Pinsky ML, Amador K, Anderson SC, Arkhipkin A, Auber A, Barri I, Bell RJ, Belmaker J, Beukhof E, Camara ML, Guevara-Carrasco R, Choi J, Christensen HT, Conner J, Cubillos LA, Diadhiou HD, Edelist D, Emblemsvåg M, Ernst B, Fairweather TP, Fock HO, Friedland KD, García CB, Gascuel D, Gislason H, Goren M, Guitton J, Jouffre D, Hattab T, Hidalgo M, Kathena JN, Knuckey I, Kidé SO, Koen-Alonso M, Koopman M, Kulik V, León JP, Levitt-Barmats Y, Lindegren M, Llope M, Massiot-Granier F, Masski H, McLean M, Meissa B, Mérillet L, Mihneva V, Nunoo FKE, O'Driscoll R, O'Leary CA, Petrova E, Ramos JE, Refes W, Román-Marcote E, Siegstad H, Sobrino I, Sólmundsson J, Sonin O, Spies I, Steingrund P, Stephenson F, Stern N, Tserkova F, Tserpes G, Tzanatos E, van Rijn I, van Zwieten PAM, Vasilakopoulos P, Yepsen DV, Ziegler P, Thorson JT (2021) Are we ready to track climate-driven shifts in marine species across international boundaries?—a global survey of scientific bottom trawl data. *Glob Change Biol* 27(2):220–236. <https://doi.org/10.1111/gcb.15404>

39. Yang H, Schell KR (2021) Real-time electricity price forecasting of wind farms with deep neural network transfer learning and hybrid datasets. *Appl Energy* 299:117242. <https://doi.org/10.1016/j.apenergy.2021.117242>
40. Durap A (2025) Machine learning-based wind speed prediction using random forest: a cross-validated analysis for renewable energy applications. *Turk J Eng* 9(3):508–518. <https://doi.org/10.31127/tuje.1624354>
41. Barone G, Buonomano A, Forzano C, Palombo A, Russo G (2023) The role of energy communities in electricity grid balancing: a flexible tool for smart grid power distribution optimization. *Renew Sustain Energy Rev* 187:113742. <https://doi.org/10.1016/j.rser.2023.113742>
42. Bishop CM (1995) *Neural networks for pattern recognition*. Oxford University Press, Oxford. <https://doi.org/10.1093/oso/9780198538493.001.0001>
43. Hornik K, Stinchcombe M, White H (1989) Multilayer feedforward networks are universal approximators. *Neural Netw* 2(5):359–366. [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)
44. Gardner MW, Dorling SR (1998) Artificial neural networks (the multilayer perceptron)—a review of applications in the atmospheric sciences. *Atmos Environ* 32(14–15):2627–2636. [https://doi.org/10.1016/S1352-2310\(97\)00447-0](https://doi.org/10.1016/S1352-2310(97)00447-0)
45. LeCun Y, Bengio Y, Hinton G (2015) Deep learning. *Nature* 521(7553):436–444. <https://doi.org/10.1038/nature14539>
46. Hochreiter S, Schmidhuber J (1997) Long short-term memory. *Neural Comput* 9(8):1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
47. Olden JD, Jackson DA (2002) Illuminating the “black box”: a randomization approach for understanding variable contributions in artificial neural networks. *Ecol Model* 154(1–2):135–150. [https://doi.org/10.1016/S0304-3800\(02\)00064-9](https://doi.org/10.1016/S0304-3800(02)00064-9)
48. Barbounis TG, Theocharis JB, Alexiadis MC, Dokopoulos PS (2006) Long-term wind speed and power forecasting using local recurrent neural network models. *IEEE Trans Energy Convers* 21(1):273–284. <https://doi.org/10.1109/TEC.2005.847954>
49. Cybenko G (1989) Approximation by superpositions of a sigmoidal function. *Math Control Signals Syst* 2(4):303–314. <https://doi.org/10.1007/BF02551274>
50. Dubey SR, Singh SK, Chaudhuri BB (2022) Activation functions in deep learning: a comprehensive survey and benchmark. *Neurocomputing* 503:92–108. <https://doi.org/10.1016/j.neucom.2022.06.111>
51. Heaton J (2018) Ian Goodfellow, Yoshua Bengio, and Aaron Courville: deep learning. *Genet Program Evolvable Mach* 19(1–2):305–307. <https://doi.org/10.1007/s10710-017-9314-z>
52. Durap A (2023) A comparative analysis of machine learning algorithms for predicting wave runup. *Anthropocene Coasts* 6(1):17. <https://doi.org/10.1007/s44218-023-00033-7>
53. Schmidhuber J (2015) Deep learning in neural networks: an overview. *Neural Netw* 61:85–117. <https://doi.org/10.1016/j.neunet.2014.09.003>
54. Nawi NM, Atomi WH, Rehman MZ (2013) The effect of data pre-processing on optimized training of artificial neural networks. *Procedia Technol* 11:32–39. <https://doi.org/10.1016/j.protcy.2013.12.159>
55. Talaei Khoei T, Ould Slimane H, Kaabouch N (2023) Deep learning: systematic review, models, challenges, and research directions. *Neural Comput Appl* 35(31):23103–23124. <https://doi.org/10.1007/s00521-023-08957-4>
56. Qi J, Du J, Siniscalchi SM, Ma X, Lee C-H (2020) On mean absolute error for deep neural network based vector-to-vector regression. *IEEE Signal Process Lett* 27:1485–1489. <https://doi.org/10.1109/LSP.2020.3016837>
57. Zafar A, Aamir M, Mohd Nawi N, Arshad A, Riaz S, Alruban A, Dutta AK, Almotairi S (2022) A comparison of pooling methods for convolutional neural networks. *Appl Sci* 12(17):8643. <https://doi.org/10.3390/app12178643>
58. Yang N, Tang H, Sun H, Yang X (2018) DropBand: a simple and effective method for promoting the scene classification accuracy of convolutional neural networks for VHR remote sensing imagery. *IEEE Geosci Remote Sens Lett* 15(2):257–261. <https://doi.org/10.1109/LGRS.2017.2785261>
59. Yang H, Liu J, Sun H, Zhang H (2020) PACL: piecewise arc cotangent decay learning rate for deep neural network training. *IEEE Access* 8:112805–112813. <https://doi.org/10.1109/ACCESS.2020.3002884>
60. Xue Z, Wang Y, Ren Y, Zhang X (2024) The robust supervised learning framework: harmonious integration of twin extreme learning machine, squared fractional loss, capped L2,p-norm metric, and fisher regularization. *Symmetry* 16(9):1230. <https://doi.org/10.3390/sym16091230>
61. Li R, Gu X, Shen Y, Li K, Li Z, Zhang Z (2022) Smart and rapid design of nanophotonic structures by an adaptive and regularized deep neural network. *Nanomaterials* 12(8):1372. <https://doi.org/10.3390/nano12081372>
62. Ang KM, El-kenawy E-SM, Abdelhamid AA, Ibrahim A, Alharbi AH, Khafaga DS, Tiang SS, Lim WH (2022) Optimal design of convolutional neural network architectures using teaching–learning-based optimization for image classification. *Symmetry* 14(11):2323. <https://doi.org/10.3390/sym14112323>
63. Chen D-W, Miao R, Deng Z-Y, Lu Y-Y, Liang Y, Huang L (2020) Sparse logistic regression with L1/2 penalty for emotion recognition in electroencephalography classification. *Front Neuroinform*. <https://doi.org/10.3389/fninf.2020.00029>
64. Nuti G, Cross A-I, Rindler P (2022) Evidence-based regularization for neural networks. *Mach Learn Knowl Extract* 4(4):1011–1023. <https://doi.org/10.3390/make4040051>

65. Zaragori T, Oster J, Roch V, Hossu G, Chawki MB, Grignon R, Pouget C, Gauchotte G, Rech F, Blonski M, Taillandier L, Imbert L, Verger A (2022) ¹⁸F-FDOPA PET for the noninvasive prediction of glioma molecular parameters: a radiomics study. *J Nucl Med* 63(1):147–157. <https://doi.org/10.2967/jnumed.120.261545>
66. Laria JC, Clemmensen LH, Ersbøll BK, Delgado-Gómez D (2022) A generalized linear joint trained framework for semi-supervised learning of sparse features. *Mathematics* 10(16):3001. <https://doi.org/10.3390/math10163001>
67. Lin S, Zeng J, Fang J, Xu Z (2014) Learning rates of l^q coefficient regularization learning with Gaussian kernel. *Neural Comput* 26(10):2350–2378. https://doi.org/10.1162/NECO_a_00641
68. Saran NA, Nar F (2025) Fast binary logistic regression. *PeerJ Comput Sci* 11:e2579. <https://doi.org/10.7717/peerj-cs.2579>
69. Chakraborty S, Morolia A, Peduri A (2023) Quantum regularized least squares. *Quantum* 7:988. <https://doi.org/10.22331/q-2023-04-27-988>
70. Chen Y, Du M, Zhang J, Zhang G, Su L, Li K, Zhao F, Yi H, Wang L, Cao X (2023) Generalized conditional gradient method with adaptive regularization parameters for fluorescence molecular tomography. *Opt Express* 31(11):18128. <https://doi.org/10.1364/OE.486339>
71. Choi H, Lee H (2023) Exploiting all samples in low-resource sentence classification: early stopping and initialization parameters. *IEEE Access* 11:30768–30782. <https://doi.org/10.1109/ACCESS.2023.3261884>
72. Niu Z, Dong H, Qin AK, Gu T (2024) FLrce: resource-efficient federated learning with early-stopping strategy. *IEEE Trans Mob Comput* 23(12):14514–14529. <https://doi.org/10.1109/TMC.2024.3447000>
73. Cao W, Tian F, Liu Z, Liu F, Zhao Y, Shi Y, Wang W, Guo X (2024) MLPCC-MLMAE-based early stopping strategy for unsupervised 3-D seismic data reconstruction. *IEEE Geosci Remote Sens Lett* 21:1–5. <https://doi.org/10.1109/LGRS.2023.3323649>
74. Shorten C, Khoshgoftaar TM (2019) A survey on image data augmentation for deep learning. *J Big Data* 6(1):60. <https://doi.org/10.1186/s40537-019-0197-0>
75. Semenoglou A-A, Spiliotis E, Assimakopoulos V (2023) Data augmentation for univariate time series forecasting with neural networks. *Pattern Recogn* 134:109132. <https://doi.org/10.1016/j.patcog.2022.109132>
76. Alsowaidan S, Al-Hurban A, Alsaber A, Anbar A (2024) Assessment of seasonal variations in the air quality index (2019–2022) in Al-Jahra city, Kuwait. *Kuwait J Sci* 51(4):100280. <https://doi.org/10.1016/j.kjs.2024.100280>
77. Moghimi A, Tavakoli Darestani A, Mostofi N, Fathi M, Amani M (2024) Improving forest above-ground biomass estimation using genetic-based feature selection from Sentinel-1 and Sentinel-2 data (case study of the Noor forest area in Iran). *Kuwait J Sci* 51(2):100159. <https://doi.org/10.1016/j.kjs.2023.11.008>
78. Combettes PL, Pesquet J-C (2020) Lipschitz certificates for layered network structures driven by averaged activation operators. *SIAM J Math Data Sci* 2(2):529–557. <https://doi.org/10.1137/19M1272780>
79. Neskornik V, Freire PJ, Napoli A, Spinnler B, Schairer W, Prilepsky JE, Costa N, Turitsyn SK (2020) Simplifying the supervised learning of Kerr nonlinearity compensation algorithms by data augmentation. In: 2020 European conference on optical communications (ECOC), pp 1–4. <https://doi.org/10.1109/ECOC48923.2020.9333417>
80. Shu K (2020) Prediction of soybean yield using self-normalizing neural networks. In: Proceedings of the 2020 5th international conference on machine learning technologies, pp 51–55. <https://doi.org/10.1145/3409073.3409092>
81. Tyass I, Khalili T, Rafik M, Abdelouahed B, Raihani A, Mansouri K (2023) Wind speed prediction based on statistical and deep learning models. *Int J Renew Energy Dev* 12(2):288–299. <https://doi.org/10.14710/ijred.2023.48672>
82. Lawal A, Rehman S, Alhems LM, Alam MdM (2021) Wind speed prediction using hybrid 1D CNN and BLSTM network. *IEEE Access* 9:156672–156679. <https://doi.org/10.1109/ACCESS.2021.3129883>
83. Domingos P (2012) A few useful things to know about machine learning. *Commun ACM* 55(10):78–87. <https://doi.org/10.1145/2347736.2347755>
84. Sinha RK (2024) Book review: Christoph Molnar. 2020. Interpretable machine learning: a guide for making black box models explainable. *Metamorph J Manag Res* 23(1):92–93. <https://doi.org/10.1177/09726225241252009>
85. Schwartz R, Dodge J, Smith NA, Etzioni O (2020) Green AI. *Commun ACM* 63(12):54–63. <https://doi.org/10.1145/3381831>
86. Strubell E, Ganesh A, McCallum A (2019) Energy and policy considerations for deep learning in NLP. In: Proceedings of the 57th annual meeting of the association for computational linguistics, pp 3645–3650. <https://doi.org/10.18653/v1/P19-1355>
87. Kang J, Parker F, Yoo CH (2007) Soil-structure interaction and imperfect trench installations for deeply buried concrete pipes. *J Geotech Geoenviron Eng* 133(3):277–285. [https://doi.org/10.1061/\(ASCE\)1090-0241\(2007\)133:3\(277\)](https://doi.org/10.1061/(ASCE)1090-0241(2007)133:3(277))
88. Kitaev N, Kaiser Ł, Levskaya A (2020) Reformer: the efficient transformer. In: International conference on learning representations (ICLR), pp 1–12
89. Amershi S, Begel A, Bird C, DeLine R, Gall H, Kamar E, Nagappan N, Nushi B, Zimmermann T (2019) Software engineering for machine learning: a case study. In: 2019 IEEE/ACM 41st international conference on software engineering: software engineering in practice (ICSE-SEIP), pp 291–300. <https://doi.org/10.1109/ICSE-SEIP.2019.00042>
90. Franchini G, Verucchi M, Catozzi A, Porta F, Prato M (2022) Biomedical image classification via dynamically early stopped artificial neural network. *Algorithms* 15(10):386. <https://doi.org/10.3390/a15100386>

91. Dufourg C, Pelletier C, May S, Lefèvre S (2024) Forecasting water resources from satellite image time series using a graph-based learning strategy. *Int Arch Photogram Remote Sens Spat Inf Sci XLVIII-2-2024*:81–88. <https://doi.org/10.5194/isprs-archives-XLVIII-2-2024-81-2024>
92. Surana AV, Pawar SE, Raha S, Mali N, Mukherjee T (2024) Ensemble fine tuned multi layer perceptron for predictive analysis of weather patterns and rainfall forecasting from satellite data. *ICTACT J Soft Comput* 15(2):3491–3496. <https://doi.org/10.21917/ijsc.2024.0487>
93. Sun Z, Song Q, Zhu X, Sun H, Xu B, Zhou Y (2015) A novel ensemble method for classifying imbalanced data. *Pattern Recogn* 48(5):1623–1637. <https://doi.org/10.1016/j.patcog.2014.11.014>
94. Lopez R, Gayoso A, Yosef N (2020) Enhancing scientific discoveries in molecular biology with deep generative models. *Mol Syst Biol*. <https://doi.org/10.15252/msb.20199198>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Ahmet Durap¹ 

✉ Ahmet Durap
ahmet.durap@medipol.edu.tr

¹ Faculty of Engineering and Natural Sciences, Istanbul Medipol University, 34181 Beykoz, Istanbul, Turkey